Unit – II
Software Requirement Analysis and Design

By: Yagnik Hathaliya

Major Learning Outcomes

- **Requirement Gathering and Analysis**
- **Software Requirement Specification(SRS)**
  - Characteristic ,Customer Requirement ,Functional Requirement
- **Software Design**
  - Characteristic of Good Software Design, Analysis v/s Design
- **Cohesion and Coupling**
  - Classification of Cohesion and Coupling
- **Data Flow Diagram**
  - Context Diagram, Level-1 DFD
- **Use Case Diagram**
- **Class Diagram**
- **Sequence Diagram**
- **Activity Diagram**

Requirement Gathering and Analysis

- Requirement Gathering and Analysis is the foundational phase of software development, where the project's objectives, needs, and expectations are identified and defined.
- This process helps ensure that the final product meets the users' and stakeholders' needs and requirements.
- Goal of this phase is to clearly understand the customer requirements and to systematically organize these requirements in a specific document.
- Done by System Analyst.
- Removing all ambiguities and inconsistencies from customer perception.
- This phase consist of the two activities:
  1. Requirements Gathering
  2. Requirements Analysis

- **Requirements Gathering**
  - This is the initial phase where all stakeholders (customers, end-users, project managers, etc.) provide their input on what the system should do.
  - Goal: to collect all relevant information from the customer regarding the product to be developed.
  - it involves interviewing the end users and customers and studying the existing documents to collect all possible information of the system.
  - Requirement Gathering Activities are:
    - Studying the existing documents
    - Interview with end users or customers
    - Task analysis
    - Scenario analysis
    - Brainstorming
    - Questionnaires
    - Group Discussions

- **Requirements Analysis:**
  - After gathering the requirements, the next step is to analyze and refine them to ensure they are clear, complete, and feasible.
  - This phase focuses on understanding the business problems, defining how the system can address them, and prioritizing requirements.
  - After the analyst has understood the exact customer requirements, he/she proceeds to identify and resolve the various requirements problems.
  - Finally, make sure that requirements should be specific, measurable, timely, achievable and realistic.
  - Output ➔ SRS (Software Requirements Specification)

Software Requirement Types

- Software Requirements can be classified into following categories:
  1. Customer Requirements
     - These refer to the needs of the end-users.
     - What are their expectations from the system in terms of usability and accessibility?
  2. System Requirements
     - These may include hardware or software specifications needed for the system to function.
  3. Functional Requirements
     - These define the specific behaviors or functions of the system.
     - What should the system do?
     - Example: User login, data processing, transaction handling.
  4. Non-Functional Requirements
     - These define the system's quality attributes, such as performance, security, scalability, and reliability.

Software Requirement Specification (SRS)

- SRS is a detailed document that describes the functional and non-functional requirements of a software system and serves as a guide for developers, testers, and stakeholders throughout the software development lifecycle.
- SRS is the output of requirement gathering and analysis activity, so we can say that it is a detailed description of the software that is to be developed and It describes the complete behavior the system.
- Once the customer agree to SRS Document the development team starts to develop the product and implement all functionalities which are specified in SRS document.
- The organization of SRS is done by the system analyst.

- **Benefits of SRS :**
  - Enhance Communication between customer and developer
  - Better Planning and Estimation
  - Quality Assurance
  - Enhanced Collaboration
  - Cost and Time Savings
  - Legal and Contractual Benefits
  - Foundation for Maintenance and Future Development
  - Customer Satisfaction
- **Important Categories of Users of SRS Document:**
  - Clients/Customers
  - System Architects/Designers
  - Software Developers
  - Testers/Quality Assurance (QA)
  - User Documentation Writers
  - Project Managers
  - Maintenance Engineers
  - Regulatory and Compliance Authorities

- **Content/Organization of SRS :**

1. Introduction and of the System
2. Overall Description of the System
3. Functional Requirements of the System
   - These define the specific behaviors or functions of the system.
   - What should the system do?
   - Example: User login, data processing, transaction handling.
4. Non-Functional Requirements of the System
   - The non-functional requirements describe the characteristics of the system that can't be expresses functionally. E.g. portability, maintainability, usability, security, performance etc.
5. Constraint of the System
   - That describes what the system should do or should not do. These are some general suggestions regarding development.
6. Index
7. Summary

- **Characteristics of a Good SRS Document:**
  - The important characteristics of a good SRS document are the following:
    1. Correct: The SRS document should be correct, that means every requirements included well in SRS.
    2. Unambiguous: The SRS document should be unambiguous, that means every requirements stated has one and only one interpretation.
    3. Complete: The SRS document should be complete, that means all requirements are completely stated in SRS.
    4. Consistent: The SRS document should be consistent, that means requirements are not conflict with any other requirements.
    5. Concise (Brief) : The SRS document should be concise.
    6. Verifiable : All requirements of the system as documented in the SRS document should be verifiable.
    7. Structured : It should be well-structured because a well-structured document is easy to understand and modify.
    8. Modifiable: The SRS document should be modifiable, that means all requirements stated In SRS can be modify easily.
    9. Portable : The SRS document should be portable.
    10. Traceable: The SRS document should be traceable, that means it can be easily traced through testing or any other method.

- **Characteristics of a Bad SRS Document :**
  - Most important problems are incompleteness, ambiguity and contradiction Following are some other category of problems that occur in many SRS documents that makes a bad SRS Document.
    - Over Specification
    - Forward References
    - Wishful Thinking
- **Problems without SRS Document :**
  - The important problems that an organization would face if it does not develop an SRS document are as follows :
    - Without developing the SRS document, the system would not be implemented according to customer needs.
    - Software developers would not know whether what they are developing is what exactly required by the customer
    - Without SRS document, it will be very much difficult for the maintenance engineers to understand the functionality of the system.
    - It will be very much difficult for user document writers to write the users' manuals properly without understanding the SRS document.

- **Customer Requirements :**
  - It Represents the need, expectations and desire of the end users or customers from the system.
  - This Should be clear, specific, unambiguous, measurable and traceable to ensure that the software meets the customer expectations.
  - How to Identify customers requirements:
    - Interview with end users or customers
    - Brainstorming
    - Questionnaires
    - Group Discussions
  - How to Write customers requirements in SRS:
    - Identify the customers
    - Gather Requirements
    - Organize Requirements
    - Use Consistent Format

- **Functional Requirements :**
  - These define the specific behaviors or functions of the system.
  - The functional requirements for a system describe the functionalities or services that the system is expected to provide.
  - A function is described as a set of inputs, process and a set of outputs.
  - Functional requirements may be calculations, data manipulations and processing, technical details or other specific functionalities that define what a system is suppose to do.
  - Functional requirements of the system are captured in use cases.
  - All functional requirements must start with a 'verb'
  - How to Identify Functional Requirements?
    - From informal problem description or from conceptual understanding of the system.
    - Identify from user perspective.
  - How to document the functional requirements?
    - Specify the input data domain, processing and output data domain.

- **Example:**

  R1: Withdraw Cash from ATM

  – Input: Enter a Valid Card

  – Description: The withdraw cash function first determines the type of account that the user has and the account number from which the user wishes to withdraw cash. It checks the balance to determine whether the requested amount is available in the account. If enough balance is available, it outputs the required cash, otherwise it generates an error message.

  – Output: Collect Required Amount

- **Example:**
  - R1: Withdraw Cash from ATM
    - Description: The withdraw cash function first determines the type of account that the user has and the account number from which the user wishes to withdraw cash. It checks the balance to determine whether the requested amount is available in the account. If enough balance is available, it outputs the required cash, otherwise it generates an error message.

R1.1 Enter Card

Input: Enter a Valid ATM Card

Output: user prompted to enter PIN.

R1.2: Enter PIN

Input: Enter a Valid PIN

Output: Showing the display with various operation.

R1.3: Select Operation Type

Input: Select withdraw amount option

Output: User prompted to enter amount

R1.4: Get Required Amount

Input: amount to be withdrawn

Output: requested cash and print receipt of the transaction

- **Example: Do it Yourself**
  - R2: Deposit Cash
  - R3: Check Balance
  - R4: Change PIN
  - R5: Transfer Money
  - Etc.…

- **Non-Functional Requirements :**
  - These define the system's quality attributes, such as performance, security, scalability, and reliability.
  - It describe the characteristics of the system that can't be expressed functionally.
  - Some Important Non-Functional Requirements are:
    - Portability
    - Reliability
    - Security
    - Data Integrity
    - Scalability
    - Usability
    - Performance
    - Availability

Functional Vs Non-Functional Requirements

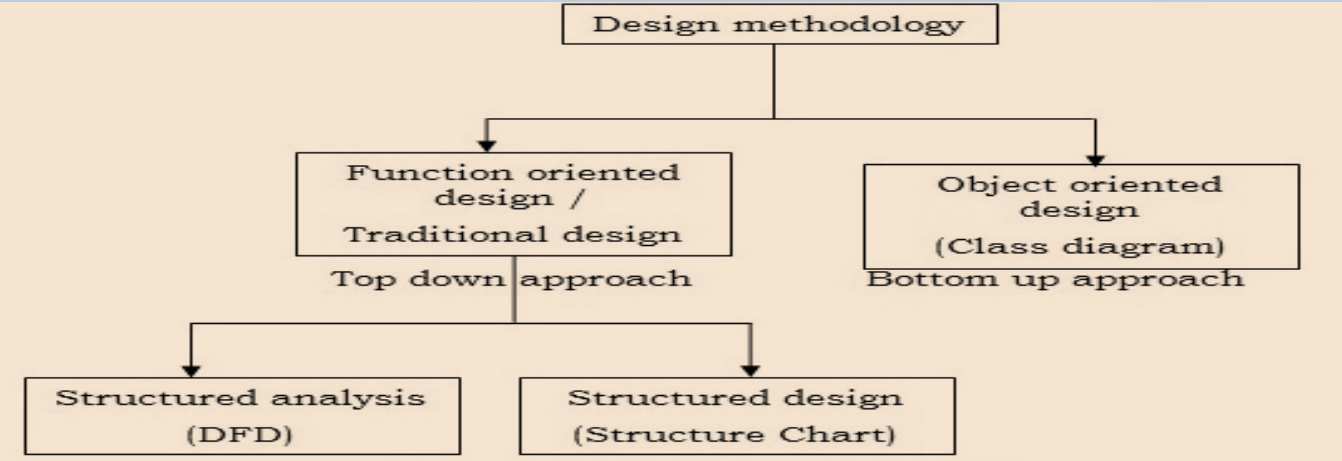
Parameter	Functional	Non-Functional
Definition	They describe the functionality and features of the system.	They describe the quality attributes, performance, and constraints of the system.
Purpose	Define the specific behaviors or functions of the system.	Define the system's operational capabilities and constraints.
Examples	User authentication (login and logout). Data processing and reporting. Integration with third-party systems. Business rules and logic.	Performance Security Usability Scalability Reliability
Focus	They focus on the user's interactions with the system.	They focus on the overall system qualities and user experience.
Documentation	Usually documented as use cases or functional specifications.	Usually documented as quality attributes

Software Design

- Software design deals with transforming the customer requirements, as described in the SRS document, into a form (a set of documents) that is suitable for implementation in a programming language.
- Purpose : Plan a solution of the Problem Specified in SRS.
- Output : Design Documents.
- The following item must be designed during the design phase :
  - User Interface Design
  - I/O Design
  - Data Design
  - Process and Program design
  - Technical Specification etc.

- Classification of Design Activities:
  - High-Level Design (HLD)
    - Objective: Define the system's overall architecture.
    - Focus: The system's major components, their roles, and interactions.
  - Mid-Level Design
    - Objective: Refine the high-level components into more detailed modules or subsystems.
    - Focus: The internal structure and interactions of specific subsystems.
  - Low-Level Design (LLD)
    - Objective: Provide detailed specifications for each module or component.
    - Focus: The internal logic, algorithms, and data structures for individual components.

- Classification of Design Methodologies:
  - Design Methodologies are followed in software development from beginning and up to the completion of the product and it is Used to provide guidelines for the design activity.
  - The nature of the design methodologies are dependent on the following factors:
    - User Requirements
    - The software Development Environment
    - The Type of the System Being Developed
    - Qualification and Training of the Development Team
    - Available Software and Hardware
  - There are many methodologies for software design for different types of problems which is depicted in above figure,



- There are Fundamentally two Different Approaches:
  - Function Oriented Design
  - Object Oriented Design
- **Function Oriented Design:**
  - In This Set of functions are described and it is built using Top-down approach also in this data in the system is centralized and shared among different functions.
  - Function oriented design further classified into : Structure analysis and Structure design.
  - Structure Analysis
    - It is used to transform a textual description into graphical form.
    - It examines the detail structure of the system.
    - It identifies the processes and data flow among these processes.
    - In structure analysis SRS is transformed into DFD.
  - Structure Design
    - Results of structured analysis are transformed into the software design.
    - In this Functions are mapped into modules.
    - Aim : Transform DFD into structure chart.

- **Object Oriented Design:**
  - In this Objects and Their Relationships are Identified and it is built using bottom-up approach.
  - Data in the system is not centralized and shared but is distributed among the objects in the system.
  - Three Main Concepts:
    - Encapsulation: Combining data and functions into a single entity
    - Inheritance: Provide Reusability
    - Polymorphism: Same context can be used for different purposes

- **Characteristics of Good Software Design :**
  - Correctness: Good design should correctly implement all the functionalities identified in the SRS document.
  - Understandability : A good design is easily understandable.
  - Efficiency : Resource should be used efficiently.
  - Maintainability : A good design should be easily manageable for change.
  - Consistency: A good design should consistent.
  - Modularity : Decomposing a problem into modules by divide and conquer in which different modules are independent which reduce complexity.
  - Completeness : A good design should be complete in terms of various components.
  - Scalability : A good design should be scalable.
  - Flexibility : A good design should be flexible to adapt changes.
  - Reliability : A good design should be reliable.
  - Testability : A good design should be testable.

Software Analysis vs Design

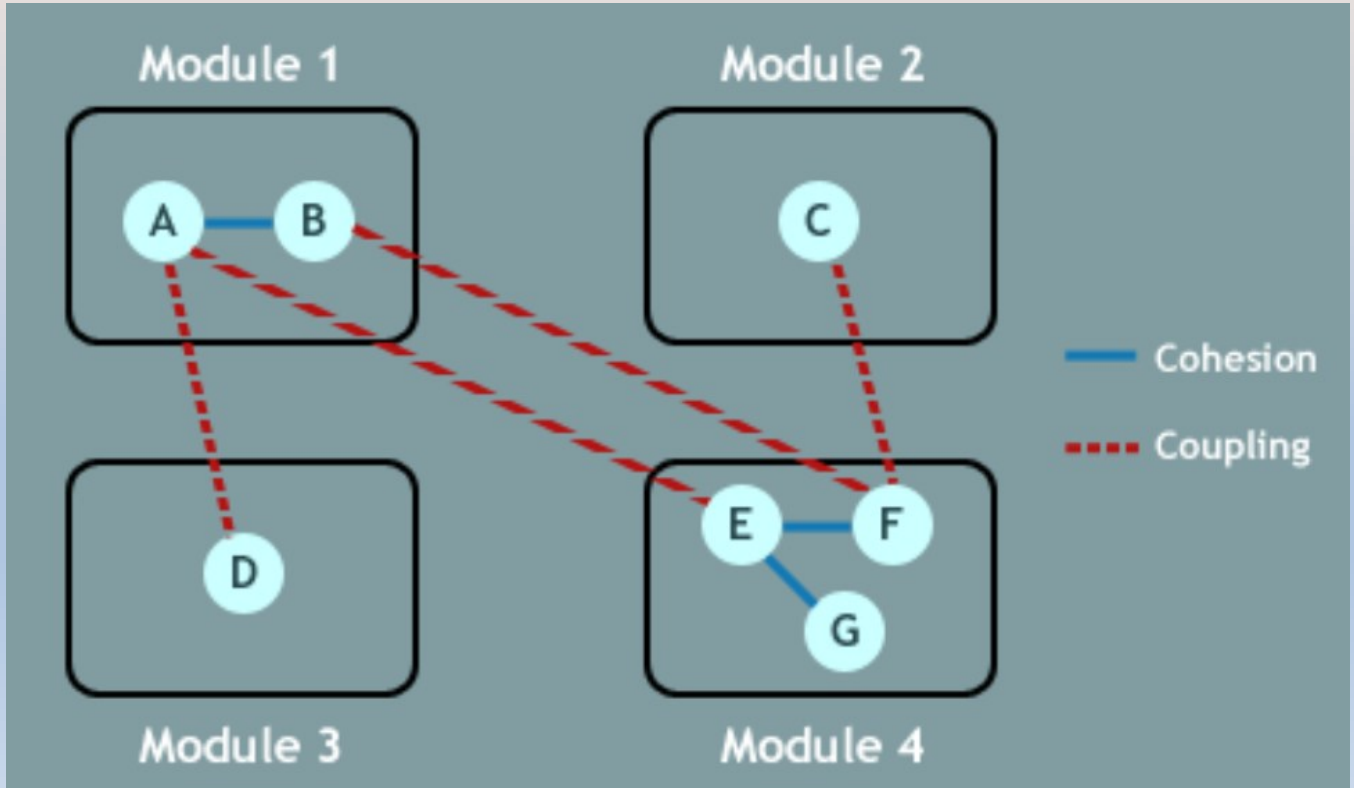
Software Analysis	Software Design
It carried out before design.	It carried out after analysis.
Focus: What the system should do (requirements)	Focus: How the system will fulfill the requirements
Output: SRS	Output: Design Specification, Architecture Diagrams, and Component Models
Activities: Requirement gathering, feasibility study, use case development	Activities: System architecture, component design, database design
Outcome: Clearly defined, documented requirements	Outcome: detailed design plan for implementation

Cohesion and Coupling Basic

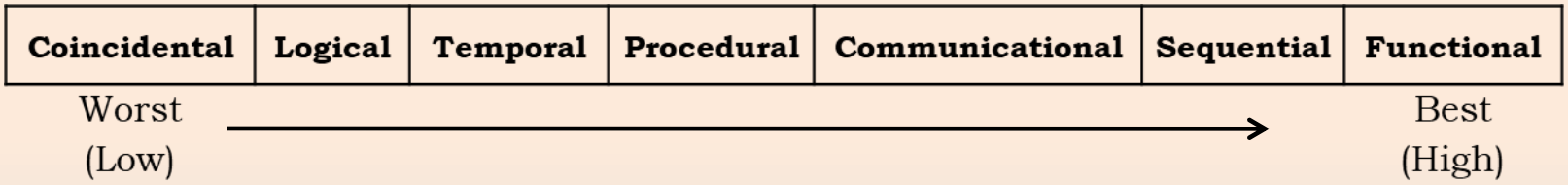
- *Software Modularity is a good property of software development.*
- *Good Software design means that clean decomposition of the problem into modules.*
- *Cohesion and Coupling are two important Modularization Criteria.*
- *High Cohesion and Low Coupling is needed for good software development.*

Cohesion

- Cohesion of a module represents how tightly bound the internal elements of a module are to one another, it is a measure of functional strength of a module and keeps the internal modules together.
- It refers to what a module can do internally, so it is also known as Inter Module Binding
- It has significant impact on quality, maintainability, and reliability of a software product.



- Classification of Cohesion:
  - The different classes of cohesion that a module may contain are shown in the following figure.



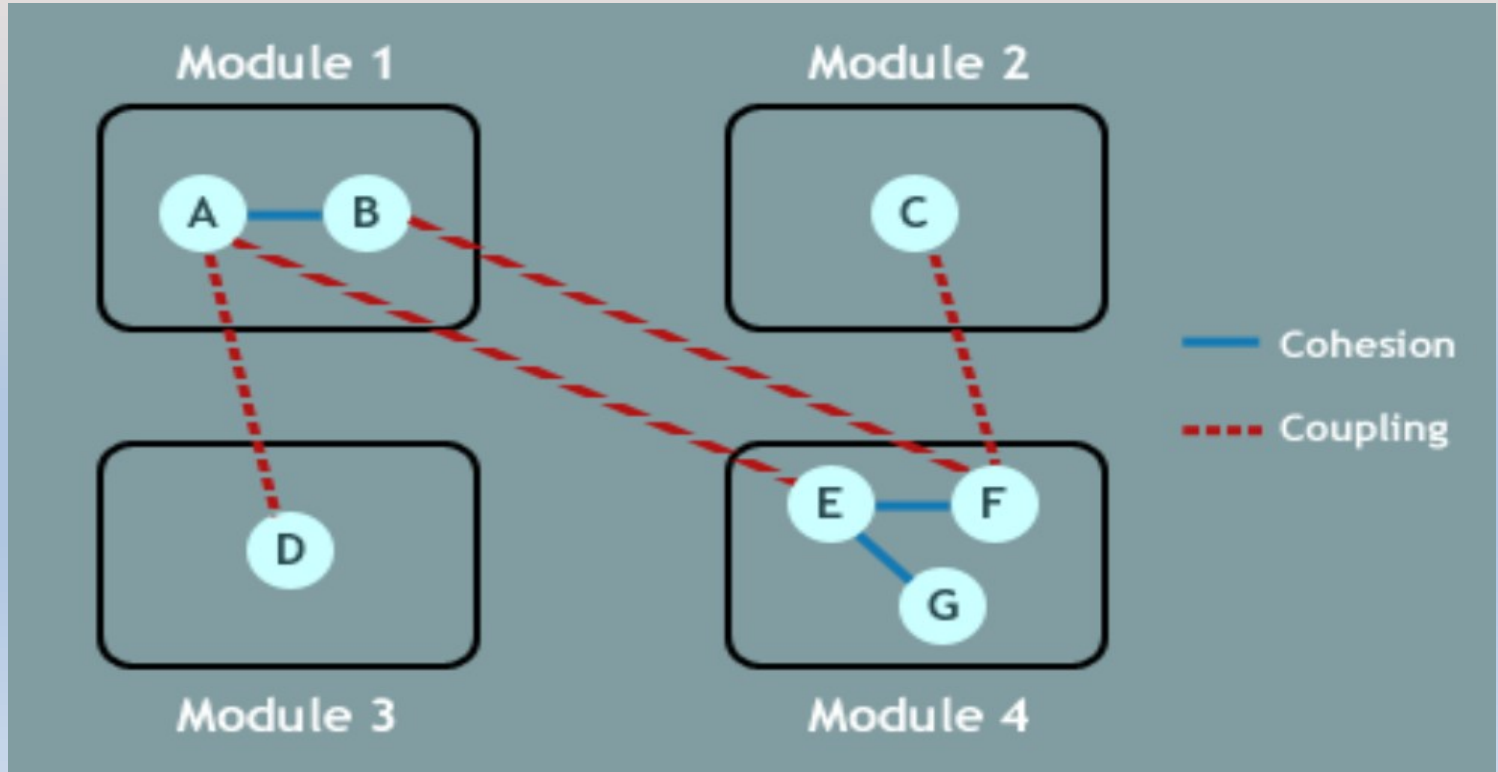
  - Coincidental Cohesion
    - A Module is in Coincidental Cohesion when there are no meaningful relationships between the elements.
    - Lowest Cohesion or Worst Cohesion.
  - Logical Cohesion
    - A module is in Logical Cohesion when there is some logical relationships between the elements of module and the elements perform functions that fall into same logical class.
    - For Example: The tasks of error handling, input and output of data.

- Temporal Cohesion.
  - A Module is in Temporal Cohesion when it is in Logical Cohesion and a module contains functions that must be executed in the same time span.
  - Example: Modules that perform activities like initialization, cleanup, startup, shut down are usually having temporal cohesion.
- Procedural Cohesion.
  - A Module is said to have Procedural Cohesion, if the set of the module are all part of a procedure (algorithm) in which certain sequence of steps are carried out to achieve an objective.
  - Example: Algorithm for decoding a message.
- Communicational Cohesion.
  - A Module is said to have Communicational Cohesion If all functions of the module refer to or update the same data structure. e.g. the set of functions defined on an array or a stack.
  - These modules may perform more than one function together.
  - Example: user authentication module in web application, doing the task of verifying user credential, retrieving user data and logging user activity, all the task are related to user authentication and focus on same data structure.

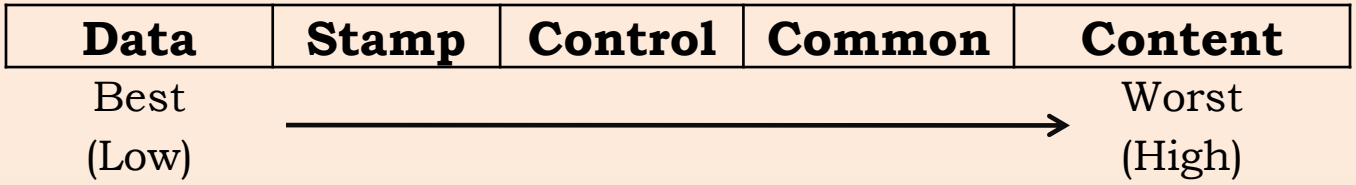
- Sequential Cohesion.
  - A Module is said to have Sequential Cohesion When the output of one element in a module forms the input to another module.
  - For Example, a function which reads data from a file and processes that data.
- Functional Cohesion.
  - Functional cohesion is the strongest cohesion.
  - A Module is said to have Functional Cohesion if all the elements of the module are related to perform a single task and achieving a single goal of a module.
  - Example: Compute square-root of the function.

Coupling

- Coupling between two modules is a measure of the degree of interaction between two modules, so coupling refers to the number of connections between 'calling' and a 'called' module.
- There must be at least one connection between them.
- It refers to the strengths of relationship between modules in a system, it indicates how closely two modules interact and how they are interdependent.
- As modules become more interdependent, the coupling increases.



- Classification of Coupling:
  - The different classes of Coupling that can exist between module are shown in the following figure.



  - Data Coupling
    - Two modules are data coupled, if they communicate using an elementary data item that is passed as a parameter between the two modules, one module passes data to another module and second module uses that data to perform its function.
    - Example: Online Shopping System where shopping cart module shares data with payment processing module.
    - It is lowest coupling and best for the software development.
  - Stamp Coupling
    - Two modules are stamp coupled, if they are connected based on a common data structure, it enables the programmer to pass an entire data structure from one module to another.
    - Example: Database Application, where different modules interact with the same data tables.

- Control Coupling
  - Two modules are control coupled, if data from one module is used to direct the order of instructions execution in another, in other term one module controls the flow of execution of another module by passing it control information.
  - Example: Secure File System, in its security module controls the flow of execution of file accessing module by passing the control information based on user authentication.
- Common Coupling
  - Two modules are common coupled, if they share data through some global data items, means two or more modules are communicating using common global data.
- Content coupling
  - Two modules are content coupled if two modules share code, e.g. a branch from one module into another module.
  - It is also known as “pathological coupling”
  - It is the highest coupling and creates more problems in software development.

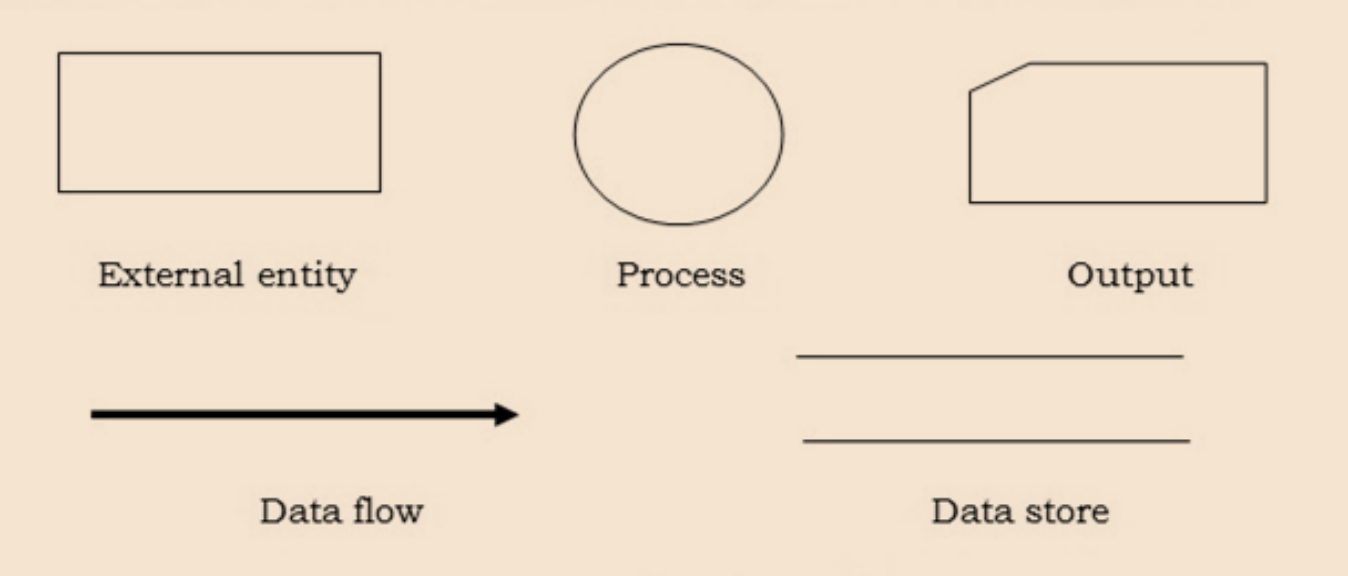
Function Oriented Software Design

- Function Oriented Software Design (FOSD) is a software development approach that focuses on decomposing the system into smaller, manageable functions or modules.
- Each function is designed to perform a specific task, contributing to the overall functionality of the system.
- This approach emphasizes the flow of data and control between functions and is often used in procedural programming languages like C.
- Thus, the system is designed from a functional viewpoint.
- Software Design is carried out using Top-Down Approach.

Data Flow Diagrams (DFDs)

- The DFD is a hierarchical graphical model of a system that shows the different processing activities or functions that the system performs and the data interchange among these functions.
- It is also known as Bubble Chart or Data Flow Graph.
- DFDs are very useful in understanding the system and can be effectively used during analysis.
- Each function is considered as a process that consumes some input data and produces some output data.
- The system is represented in terms of the input data to the system, various processing carried out on these data, and the output data generated by the system.

- Symbols Used in Construction of DFD:



  - Process:
    - It is s represented by circle or bubble and circles are annotated with names of the corresponding functions.
    - A process shows the part of the system that transforms inputs into outputs.
    - The process is named using a single word that describes what the system does functionally.
    - Generally, Process named using ‘verb’

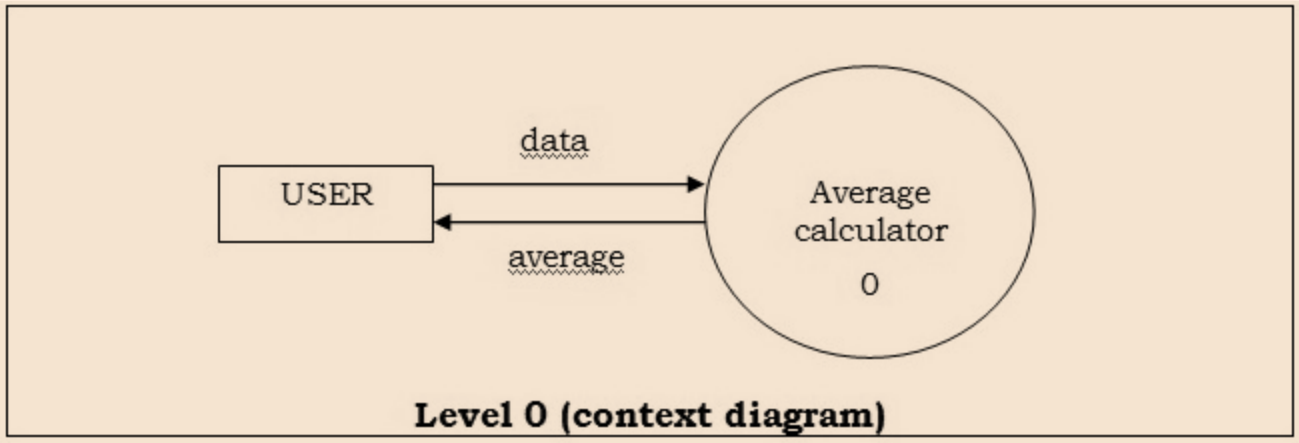
- External Entity:
  - Entity is represented by a rectangle.
  - Entities are external to the system which interacts by inputting data to the system or by consuming data produced by the system.
  - It can also define source (originator) or destination (terminator) of the system.
- Data Flow:
  - Data flow is represented by an arrow and It used to describe the movement of the data.
  - It represents the data flow occurring between two processes, or between an external entity and a process.
  - Generally, it is names using ‘noun’
- Data Store:
  - Data store is represented by two parallel lines.
  - It is generally a logical file or database, where data is stored.
  - It can be either a data structure or a physical file on the disk.
- Output:
  - Output is used when a hardcopy is produced.
  - It is graphically represented by a rectangle cut either a side

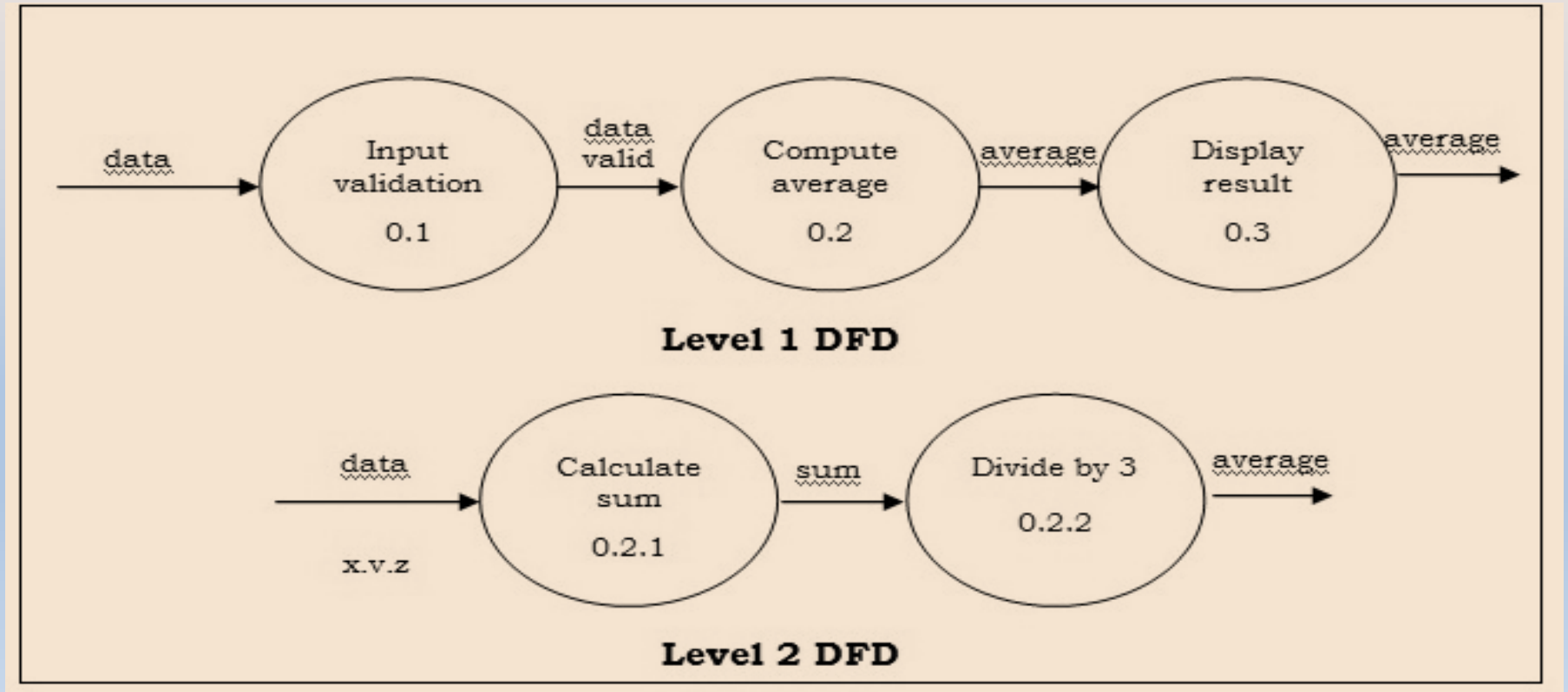
- Developing DFD Model of the System:
  - DFD starts with the most abstract level of the system (lowest level) and at each higher level, more details are introduced.
  - To develop higher level DFDs, processes are decomposed into their sub functions.
  - The abstract representation of the problem is also called Context Diagram.
- Context Diagram (Level 0 DFD):
  - The context diagram is top level diagram and it is the most abstract data flow representation of a system.
  - It only contains one process node that generalizes the function of entire system with external entities. (It represents the entire system as a single bubble.)
  - Data input and output are represented using incoming and outgoing arrows.

- Level 1 DFD:
  - To develop the level 1 DFD, we have to examine the high-level functional requirements.
  - It is recommended that 3 to 7 functional requirements can be directly represented as bubbles in 1st level.
- Further Decomposition:
  - The bubbles are decomposed into sub-functions at the successive levels.
  - Decomposition of a bubble is also known as factoring or exploding a bubble.
  - It's not a rule that particular number of levels are needed for the system.
- Numbering the Bubbles:
  - It is necessary to number the different bubbles occurring in the DFD.
  - These numbers help in uniquely identifying any bubble in the DFD by its bubble number.
  - The bubble at the context level is usually assigned the number 0 to indicate that it is the 0 level DFD. Bubbles at level 1 are numbered, 0.1, 0.2, 0.3, etc, etc.
  - When a bubble numbered x is decomposed, its children bubble are numbered x.1, x.2, x.3, etc.

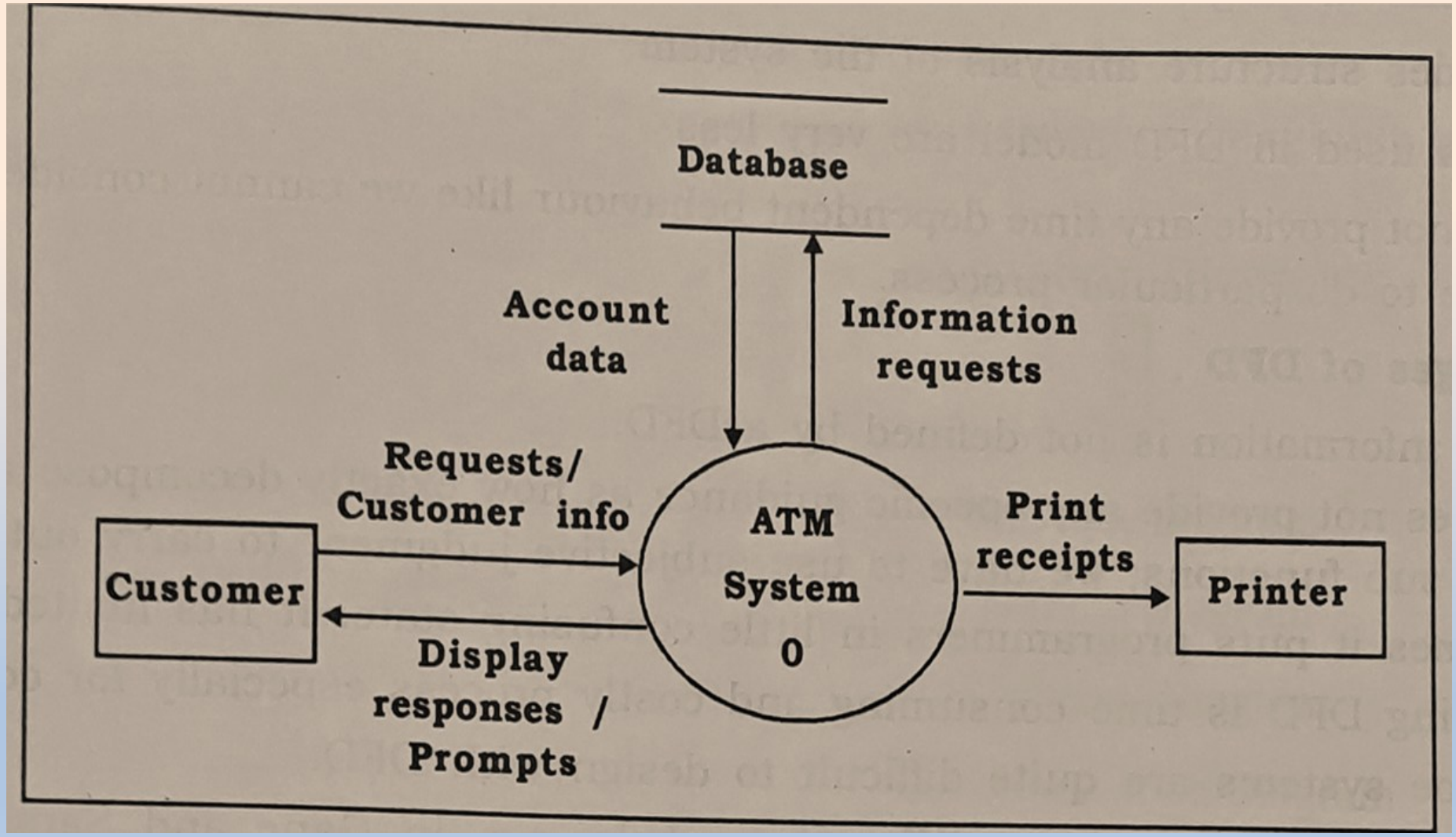
- Some Care Should be Taken While Constructing DFDs (Guidelines When Drawing DFDs):
  - A process must have at least one input and one output data flow.
  - No control information (If-THEN-ELSE) should be provided in DFD.
  - A data store must always be connected with a process.
  - A data store cannot be connected to another data store or to an external entity.
  - Data flows must be named.
  - Data flows from entities must flow into processes, and data flows to entities must come from processes.
  - There should not be detailed description of process in context diagram.
  - Each low level DFD must be balanced to its higher level DFD (input and output of the process must be matched in next level).
  - No need to draw more than one bubble in context diagram.
  - Generally, all external entities interacting with the system should be represented only in the context diagram.
  - Be careful with number of bubbles in particular level DFD, as too less or too many bubbles in DFD is oversight.
  - All the functionalities of the system specified in SRS must be captured by the DFD model.

- Example: Average Calculator for Three Numbers

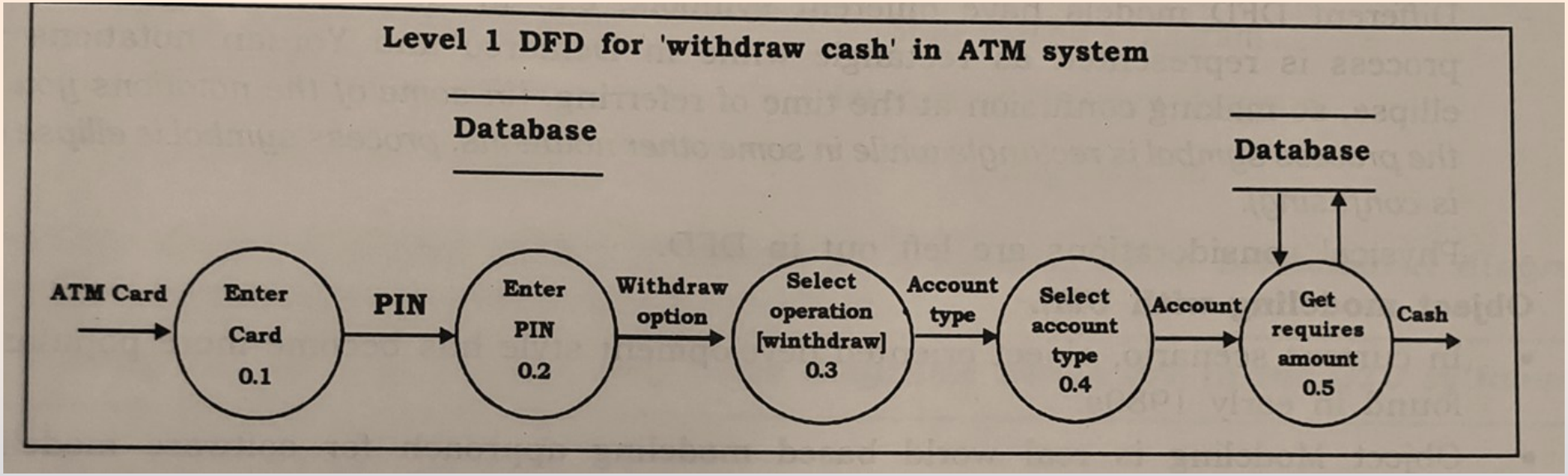


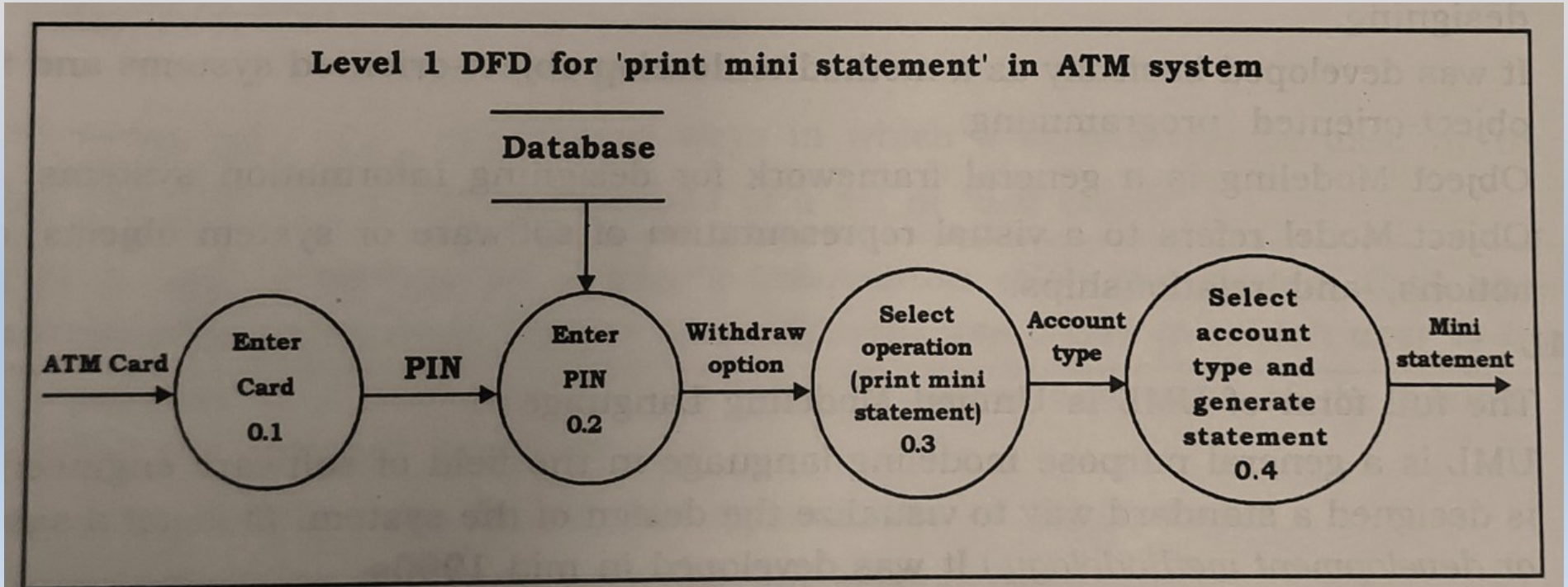


- Example: ATM System Level-0



- Example: ATM System Level-1

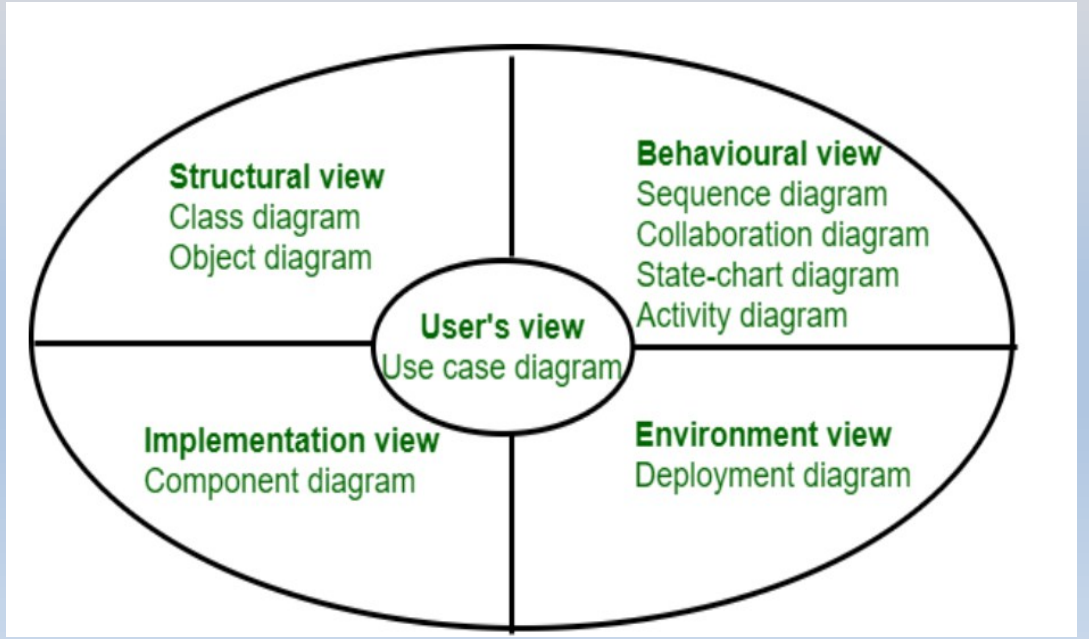




- Advantages of DFD:
  - Simple to understand and easy to use.
  - DFD can provide detailed description of the system components.
  - It provides clear understanding to the developers about the system boundaries.
  - Symbols used in DFD model are very less.
- Disadvantages of DFD:
  - Control information is not defined by a DFD.
  - No specific guidance for exact decomposition,
  - Sometimes it puts programmers in little confusing state.
  - Different models of DFD have different symbols.

Object Modelling with UML

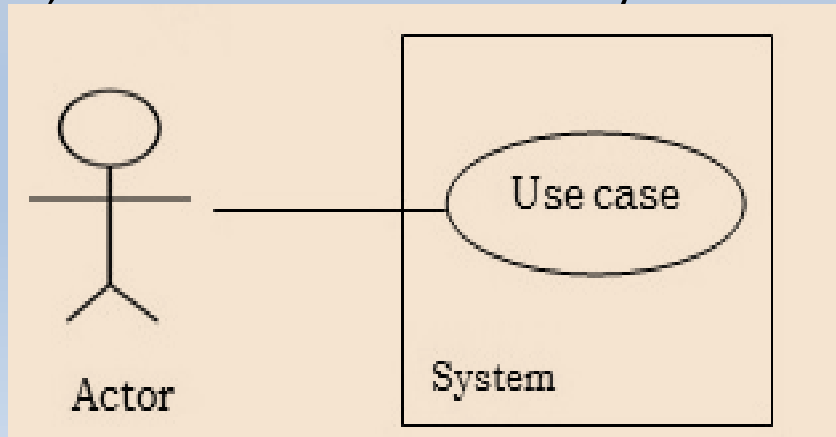
- Object modeling with UML (Unified Modeling Language) involves creating visual representations of a system's structure and behavior using a standardized set of diagrams.
- UML helps in understanding, designing, and documenting software systems.
- UML making system easy to understand using a smaller number of primitive symbols.
- It provides a set of notations (e.g. rectangles, lines, ellipses, etc.) to create a visual model of the system, UML can be used to construct nine different types of diagrams to capture five different views of a system.



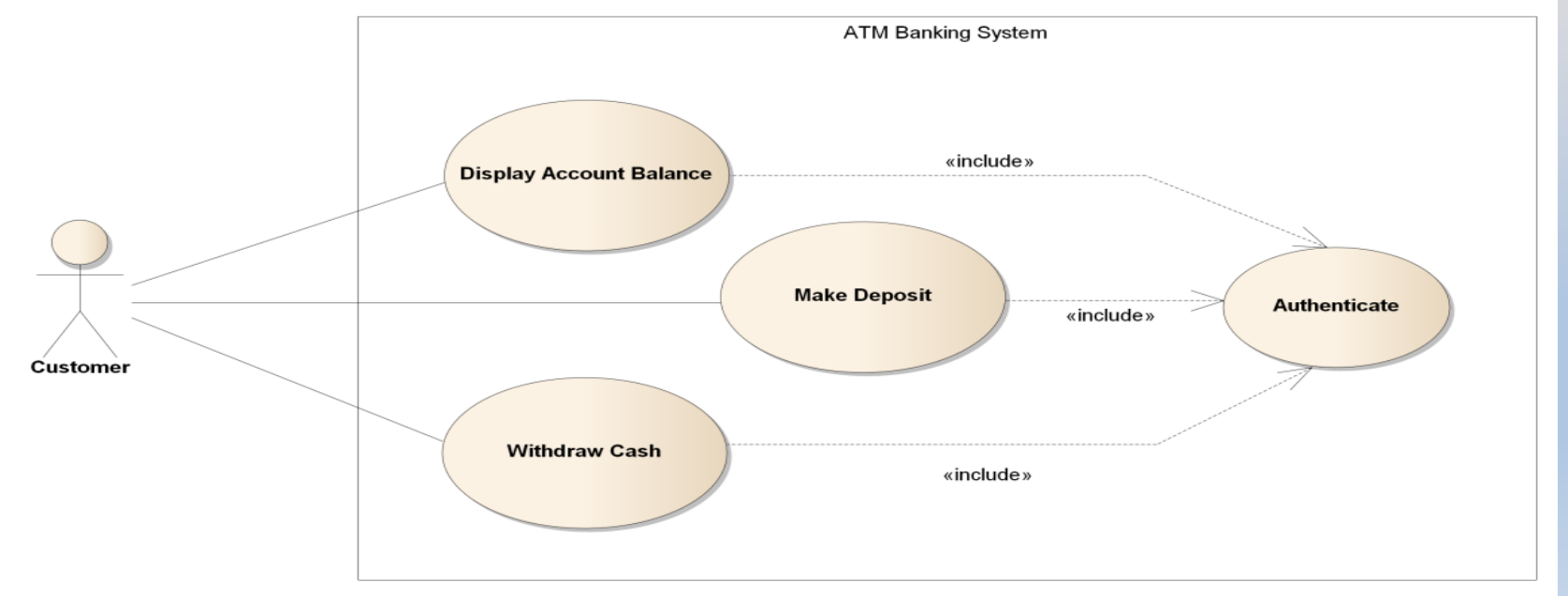
Use Case Diagram

- Use Case Diagram is a visual representation of the functional requirements of a system that shows the interactions between actors (users or other systems) and the system itself, capturing the different ways the system is used.
- Use case model of the system consists of a set of use cases and it represent the different ways in which a system can be used by the users.
- The purpose of a use case is to define the logical behavior of the system without knowing the internal structure of it.
- A simple way to find all the use cases of a system is to ask the question: "What the users can do using the system?"
- Example: In an ATM transaction system can have following use cases.
  - Check balance
  - Deposit funds
  - Withdraw cash
  - Transfer Money
  - PIN Change

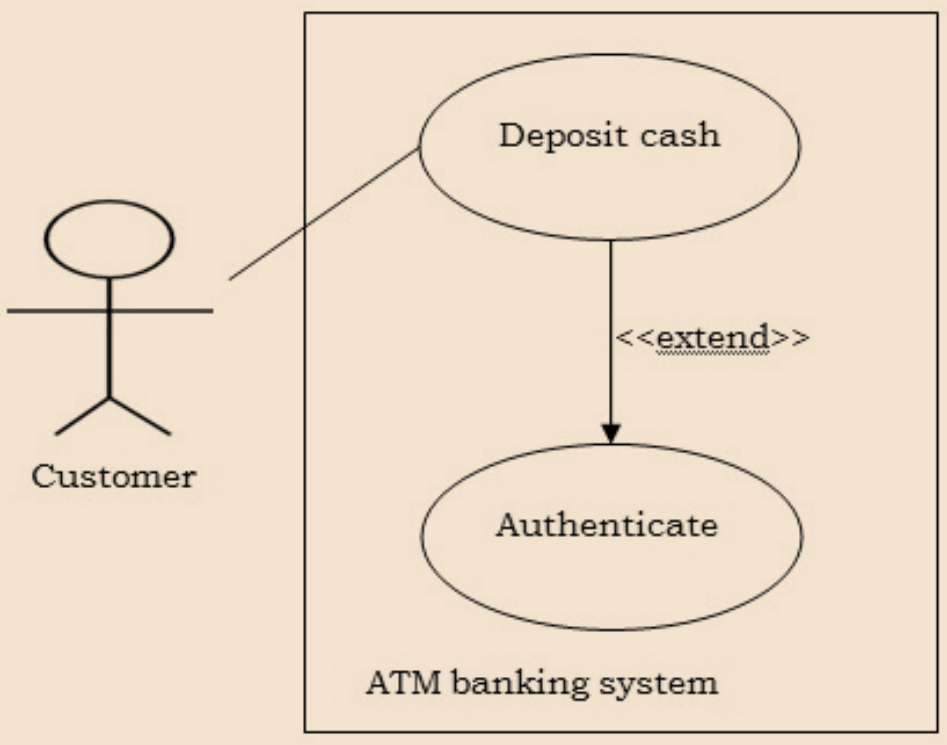
- **Component of Use Case Diagram:**
  - Use Case
    - Represented by an ellipse with the name of the use case written inside the ellipse named by a 'verb'.
    - Describe specific functionalities or services the system provides.
    - All the use cases are enclosed with a rectangle representing system boundary and that rectangle contains the name of the system.
  - Actor
    - An actor is anything outside the system that interacts with it, named by 'noun'.
    - Actors are represented by using the stick person icon.
    - An actor may be a person, machine or any external system.
    - Actors are connected to use cases by drawing a simple line connected to it.
    - Each actor must be linked to a use case, while some use cases may not be linked to actors.



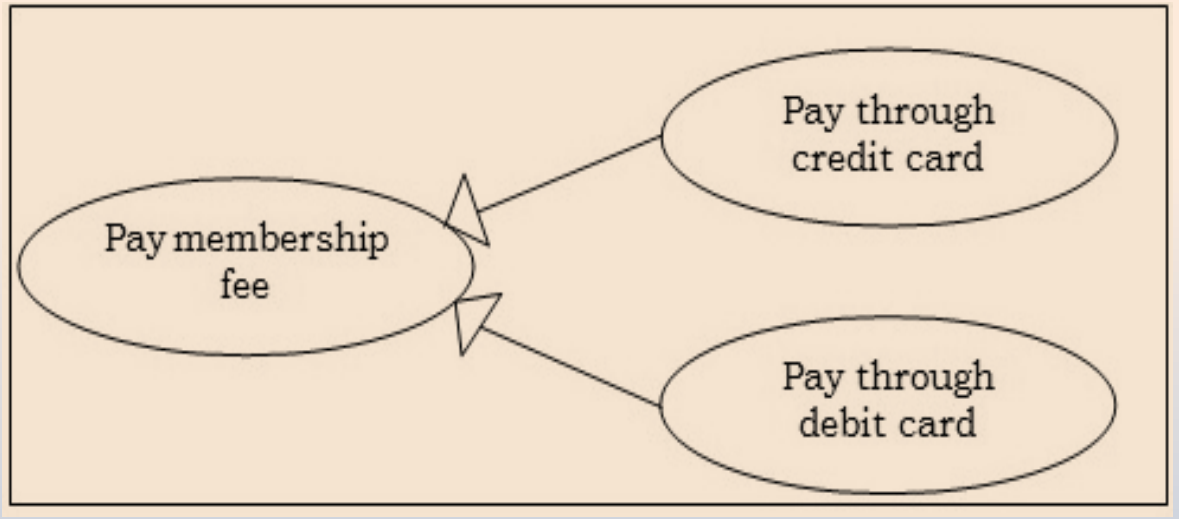
- Relationship
  - The line connecting the actor and the use case is called relationship.
  - Different relationships in use case diagram are explained below:
  - Association:
    - It is the interface between an actor and a use case and it is represented by joining a line from actor to use case
  - Include Relationship:
    - A dotted arrow with the label <<include>> pointing from one use case to another, indicating mandatory inclusion.
    - Example of include relationship of ATM



- Extend Relationship:
  - It shows optional behavior of the system.
  - A dotted arrow with the label <<extend>> pointing to a use case that extends the behavior of another.
  - Extend relationship exists when one use case calls another use case under certain condition (like: If – else condition).



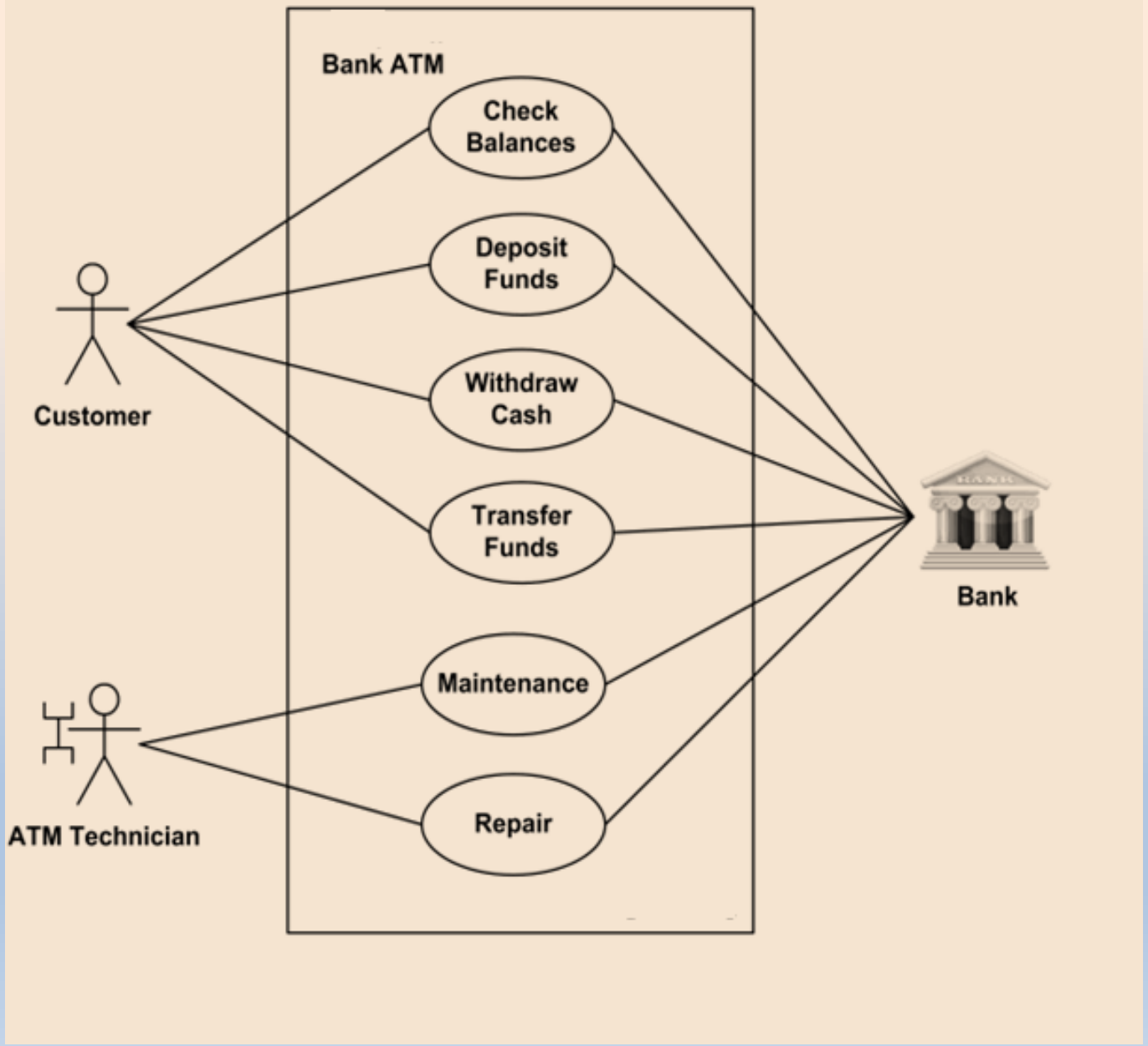
- Generalization:
  - An arrow indicating inheritance or specialization between actors or use cases and used when you have one use case that is similar to another, but does slightly different.



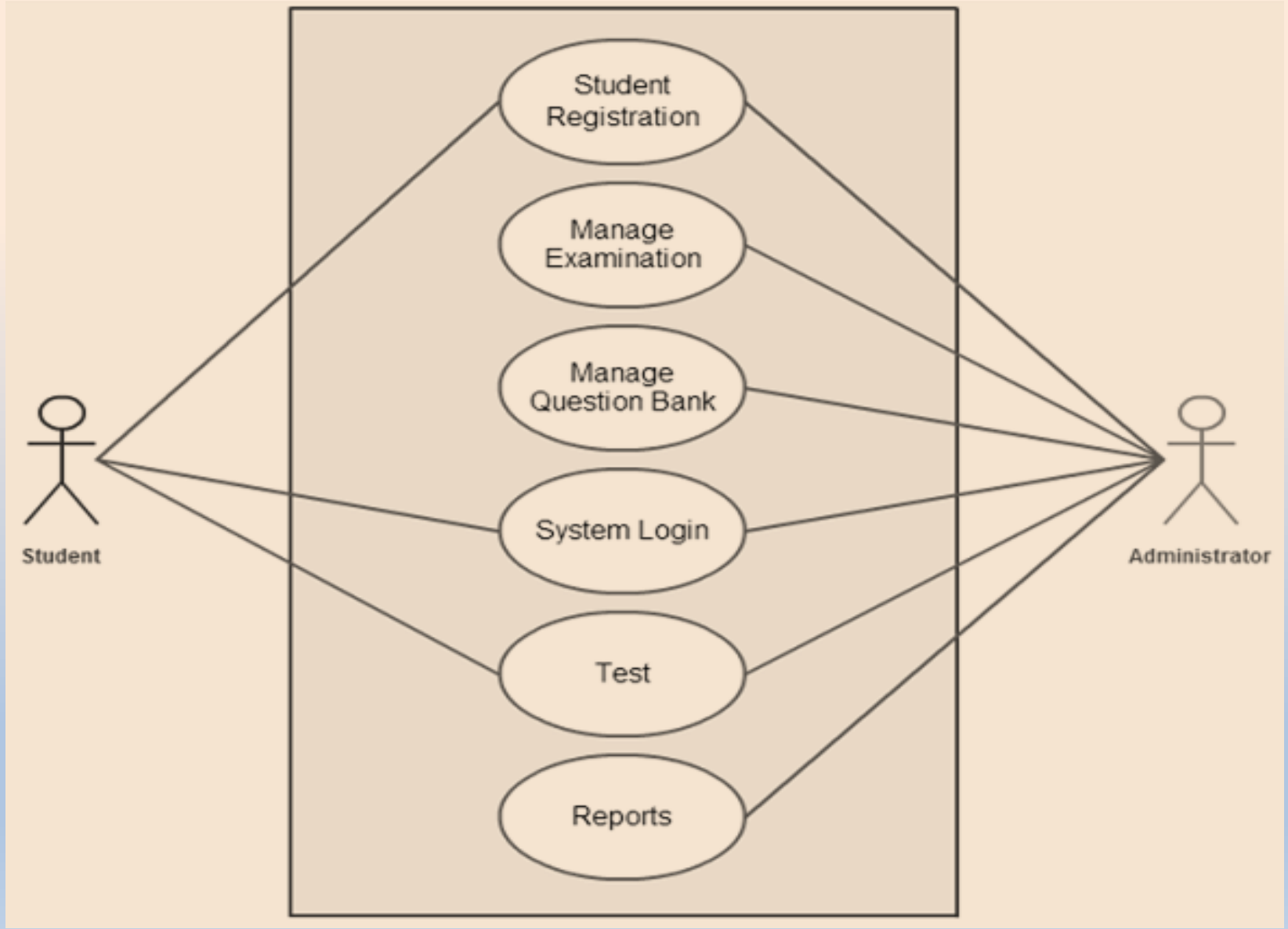
- Use Case Diagram Preparation Guidelines:
  - Identify all different users and give suitable names.
  - For each user, identify tasks, these tasks will be the use cases.
  - Use case name should be user perspective.
  - Show relationships and dependencies.

- Steps to Create a Use Case Diagram:
  - Identify Actors
  - Identify Use Cases
  - Draw System Boundary
  - Connect Actors to Use Cases
  - Add Relationships

- Example: Use case diagram of ATM:



- Example: Use case diagram of College Registration System:



- Advantages of Use Case Diagram:
  - Clear Communication
  - User-Centric Approach
  - Simplifies Complex Systems
  - Easy to Understand
  - Ensure that all functional requirement is identified

- Disadvantages of Use Case Diagram:
  - Lack of Detail
  - Not Suitable for All Systems
  - Not fully object oriented
  - Create Ambiguity

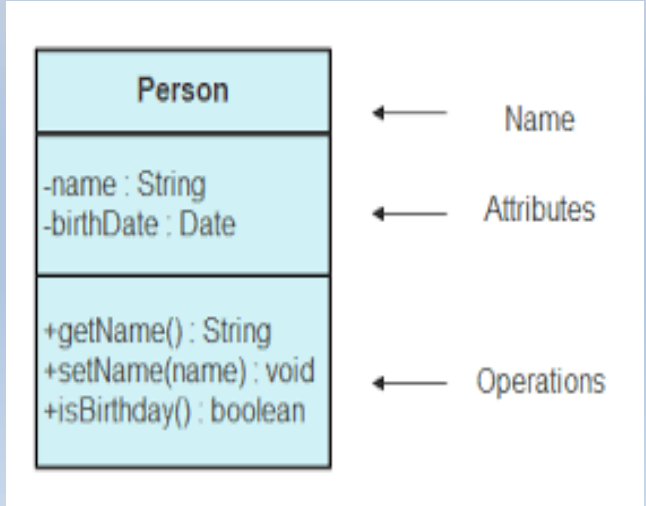
- Application of Use Case Diagram:
  - Requirement Gathering and Analysis
  - Project Management
  - Documentation and Communication
  - Testing and Validation

Class Diagram

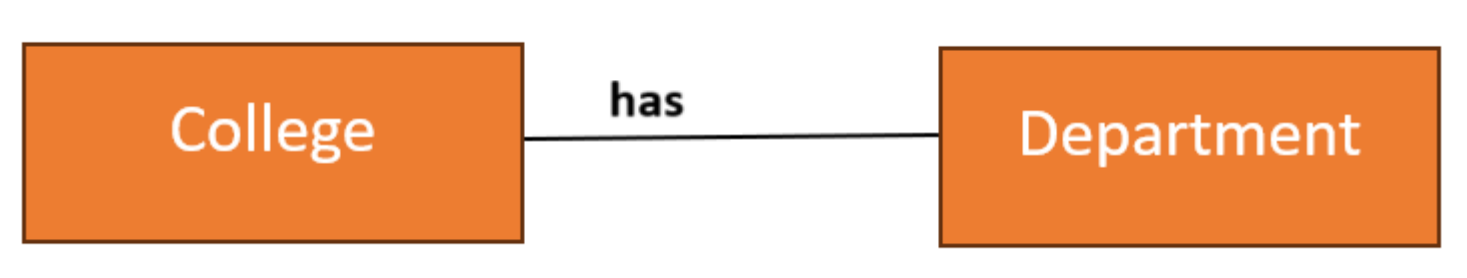
- Class Diagram is a fundamental component of UML that describes the static structure of a system by showing its classes, attributes, methods, and relationships between the classes.
- It is used to model the blueprint of a system, representing the system's objects and their interactions.
- It shows how a system is structured rather than how it behaves.
- Class diagrams can be used for the following purposes:
  - To describe the static view of a system/application
  - To describe the functionalities performed by the system.
  - To construct the software application using object-oriented languages.
  - It can be used as a base for component and deployment diagrams.
- Component of Class diagrams:
  - Set of Classes
  - Set of Relationship between classes.

- Class
  - Class is a group of objects all with common structure and behavior.
  - It is used to create object.
  - In a class diagram class is represented as rectangles, each with three sections:

1. Class Name
   - The Name of the class appears in the first partition.
   - Class name should be start with Capital Letters and place them in the center part.
2. Class Attributes
   - Attributes are shown in the second partition, that describe properties of the class.
   - Attribute are written along with visibility factors which are public(+), private (-), and protected(#).
   - Attribute type is shown after the colon.
3. Class Operations or Methods
   - Operations are shown in the third partition, they are services that the class provides.
   - Methods are represented in the form of a list, and it demonstrate how a class interacts with data.
   - The return type of a method is shown after the colon at the end of the method signature.



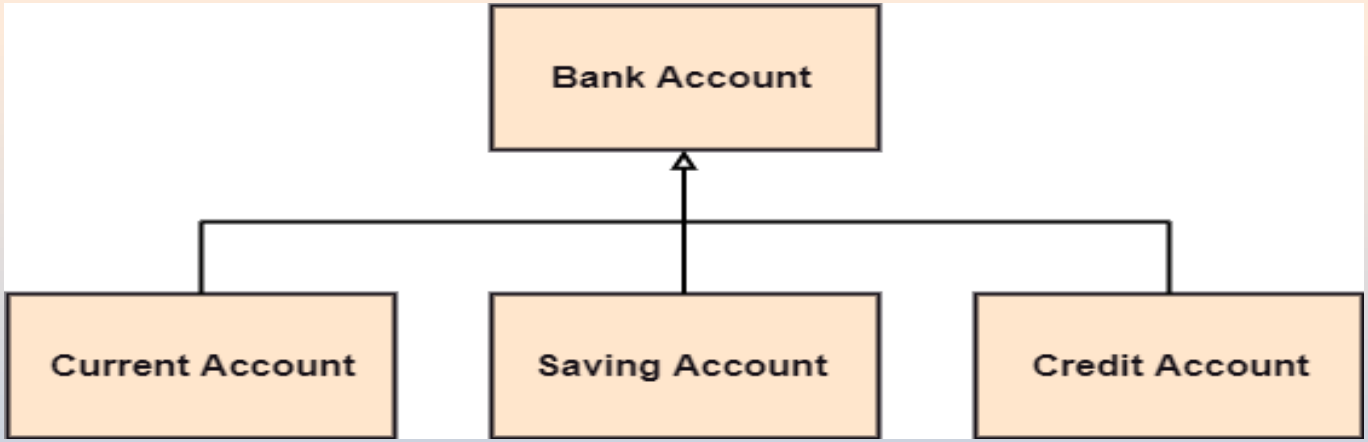
- Class Relationships:
  - It is a connection between two or more classes, and describe how the classes are related to each other in terms of functionality, behavior or data exchange.
  - Association
    - It describes a static connection between two or more classes, it depicts how many objects are there in a relationship.
    - It is represented using solid line, we can also give a name to association on the connecting line with a 'verb'.
    - Example:



- Class Relationships:
  - Multiplicity
    - It specifies how many objects of one class are associated with how many objects of another class.
    - If it is not specified by default value is 1.
    - Example: many students are studying in college.

  - Dependency
    - It is a semantic relationship between two or more classes where a change in one class cause changes in another class.
    - It is represented using a dashed arrow.
    - Example: students name is depended on his id.

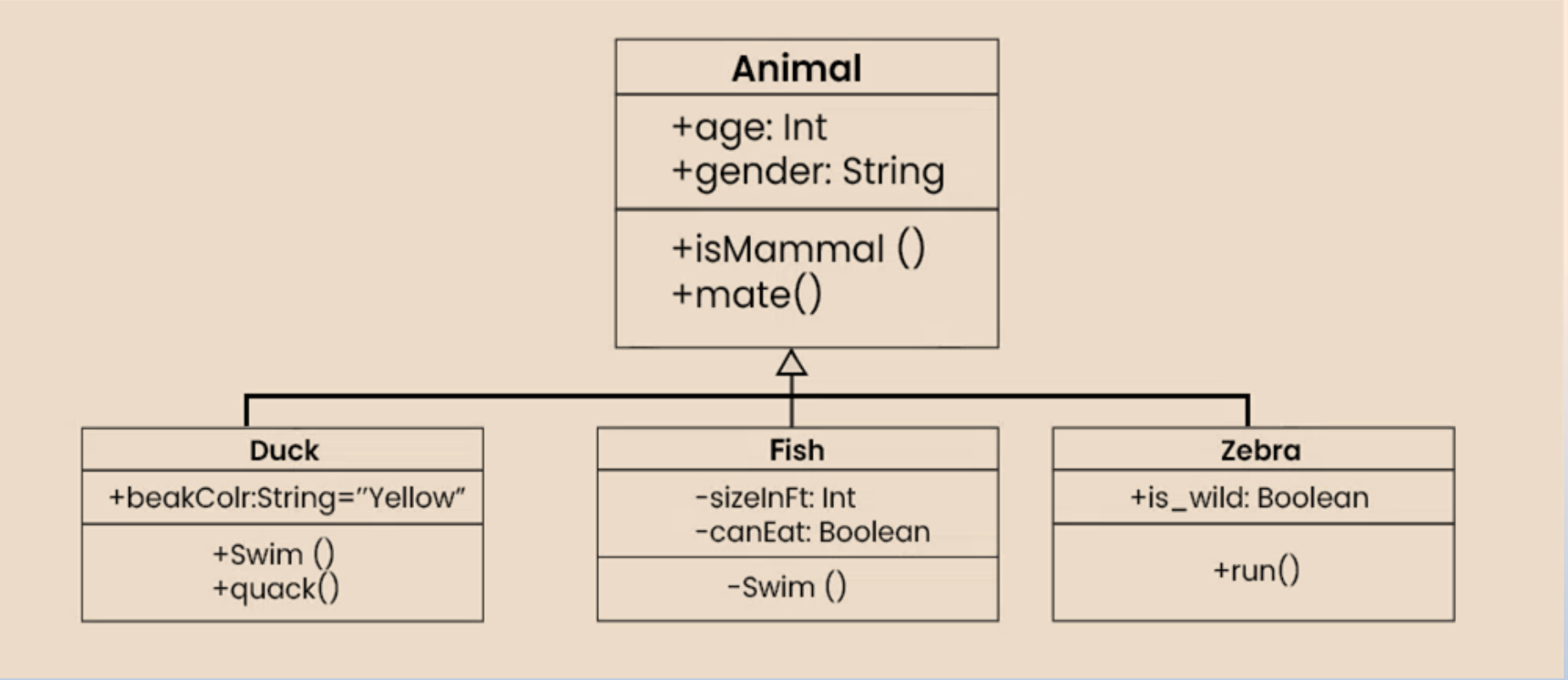
- Class Relationships:
  - Generalization
    - It is a relationship between a superclass and a child class, and shows that the subclass inherits the properties and methods of the superclass.
    - Example:



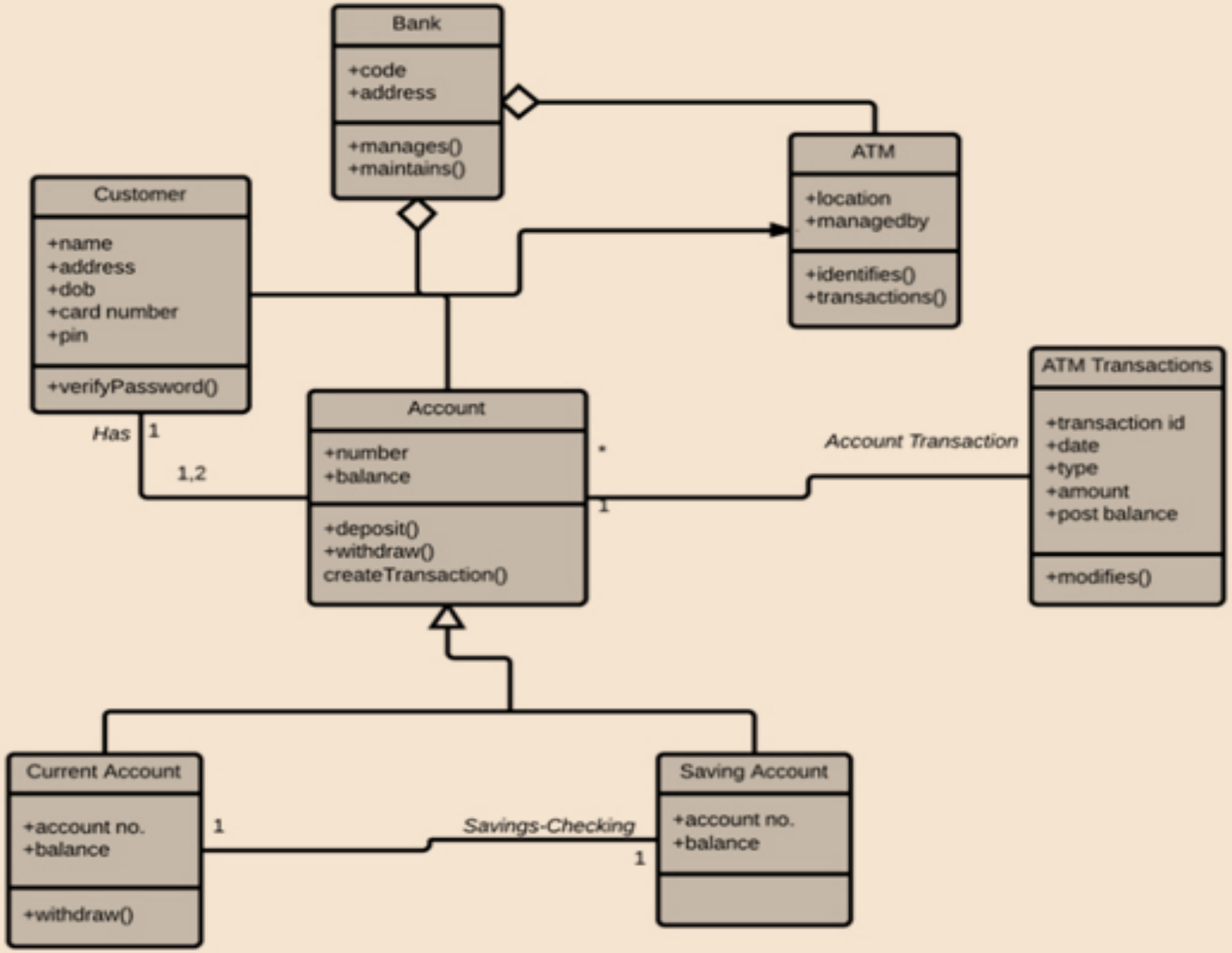
- Class Relationships:
  - Aggregation
    - It is a type of association that represents a part-whole or part-of relationship between two classes, and it demonstrates ‘consist-of’ hierarchy.
    - Example: Compony is made up of one or more employees.

  - Composition
    - It is a subset of aggregation which denotes strong relationship between two classes when one class is a part of another class.
    - Example: if all contact are removed then contact book is no longer usable.

- Example:


Animal
+age: Int
+gender: String
+isMammal ()
+mate()
Duck
+beakColr:String="Yellow"
+Swim ()
+quack()
Fish
-sizeInFt: Int
-canEat: Boolean
-Swim ()
Zebra
+is_wild: Boolean
+run()

- Example: ATM System

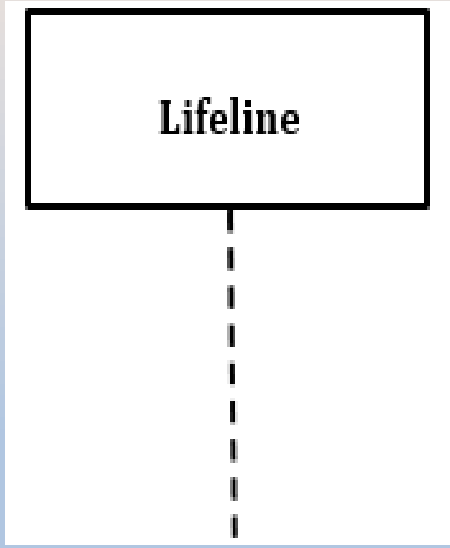


- Advantages of Class Diagram:
  - Clearly describe static view and functionalities of the system.
  - Help in construction of Software application using OO approach
  - Quality and Efficiency Improved
- Disadvantages of Class Diagram:
  - Time Consuming
  - Complexity in Large Systems
  - Changes are tedious
- Application of Class Diagram:
  - System Design and Architecture
  - Database Design
  - Code Generation
  - Testing

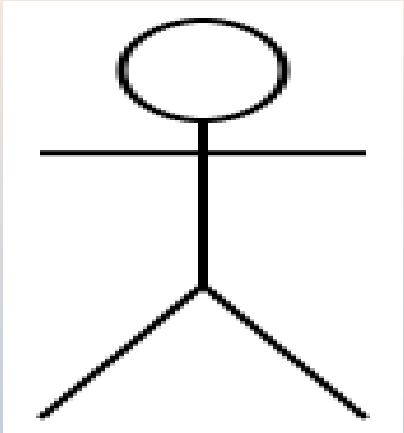
Sequence Diagram (Event Diagram)

- Sequence diagram is a type of interaction diagram in UML (Unified Modeling Language) that shows the interactions between objects in ordered sequence.
- It is used to visualize the flow of message, events, or action between the different elements in the system.
- Purpose of sequence diagram:
  - To model high level interactions among active objects
  - Understanding System Behavior
  - Clarifying Requirements
  - Documenting System Processes
  - Facilitating Communication

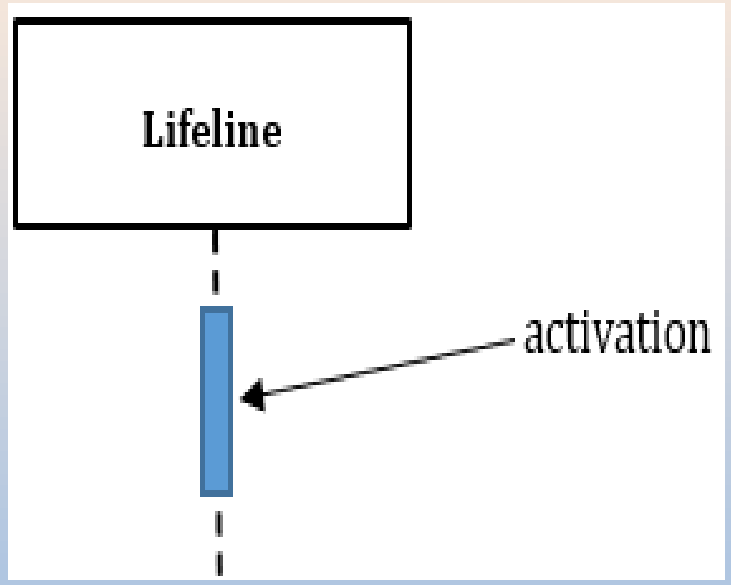
- Components (Notations) of Sequence Diagram:
  - Lifeline:
    - It is individual participant (objects) that is located at the top of the diagram.
    - Represent the existence of an object over time during the interaction.
    - Each lifeline is represented by a vertical dashed line that extends down the length of the diagram and a rectangle box at the top of line is labelled with the name of the object or component it represents.



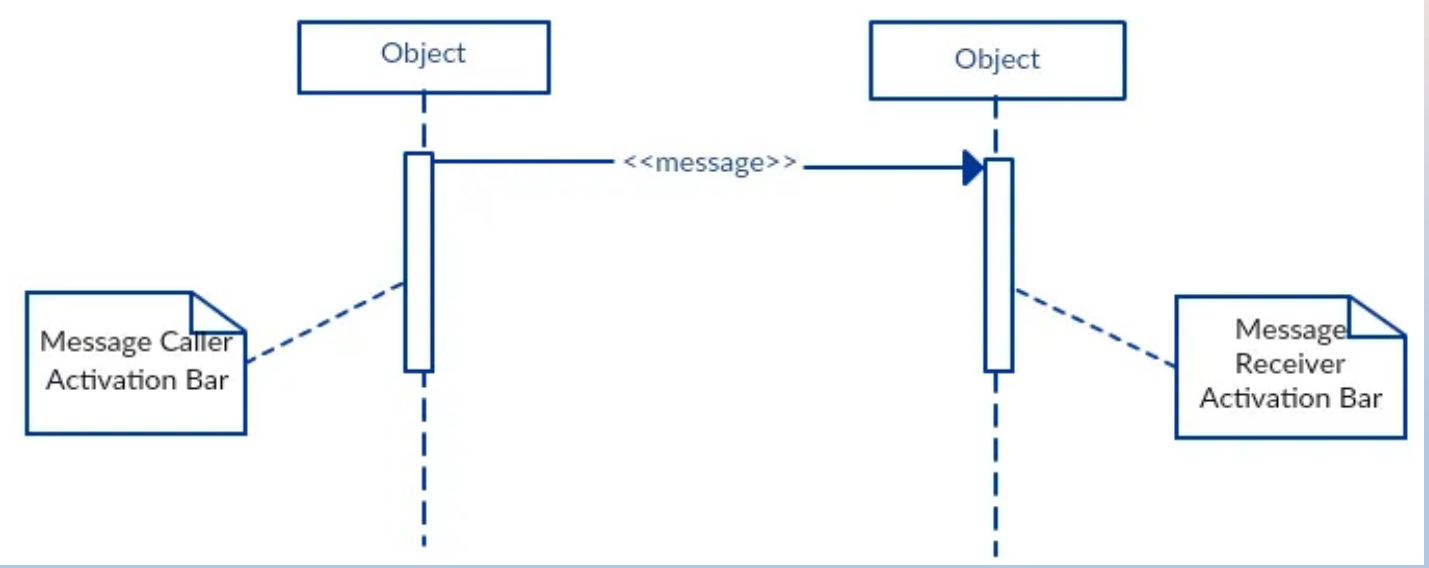
- Components (Notations) of Sequence Diagram:
  - Actor:
    - Actor represents a user, system, or external entity that interacts with the system.
    - Actors are represented as stick person icon outside the system boundary, and they are connected to the system by a dashed line.



- Components (Notations) of Sequence Diagram:
  - Activation:
    - Represent the period during which an object is performing an action or method.
    - It is runtime behavior of the object.
    - It is represented by a thin rectangle on the lifeline.



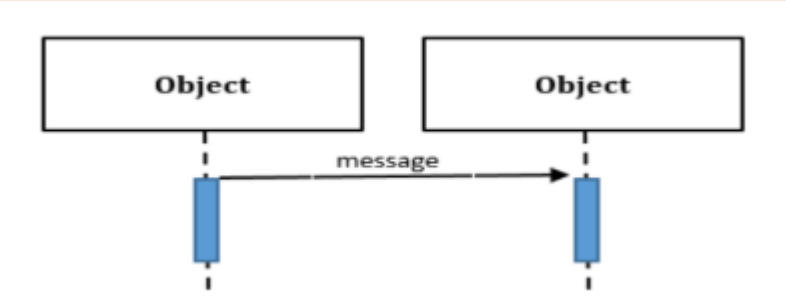
- Components (Notations) of Sequence Diagram:
  - Message:
    - Message represents the communication between objects of a system.
    - Used to convey the information from one object to another, typically, in case of request or response.
    - Messages are represented by arrows that connect the lifelines of objects, the arrowhead indicates the direction of the message, and the label on the arrow indicates the type of message being sent.



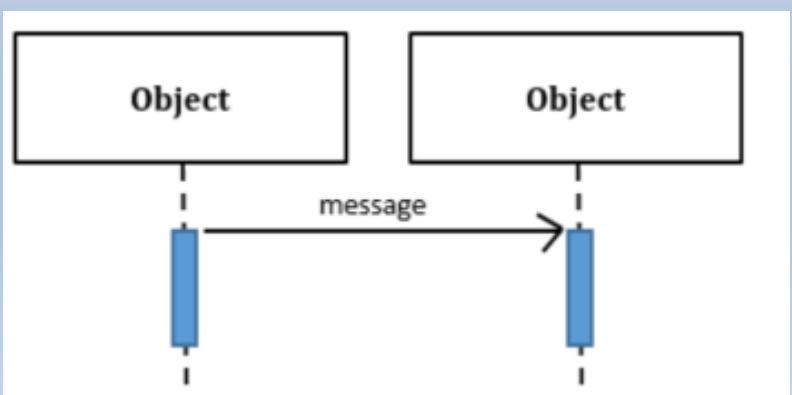
– Different Types of Message:

➢ Call Message:

- It defines a particular communication between Lifelines of an interaction.
- Call message is of two types:
  - Synchronous Call Message (it requires a response from the receiving object), that is represented using filled arrow head.

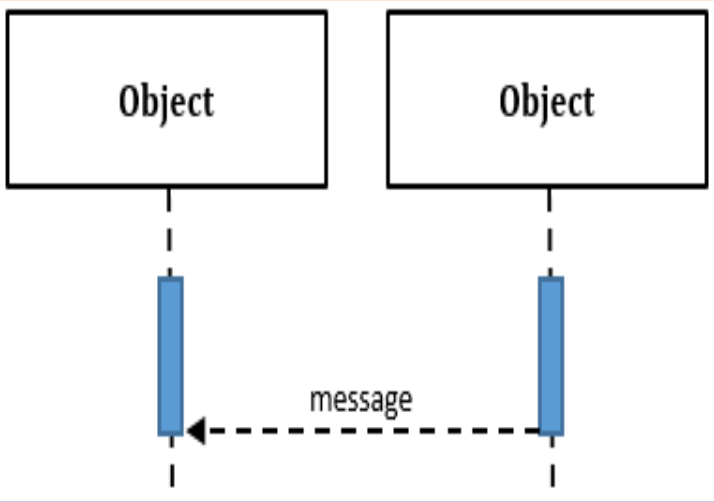


  - Asynchronous Call Message (it does not require an immediate response from the receiving object), that is represented using open arrow head.



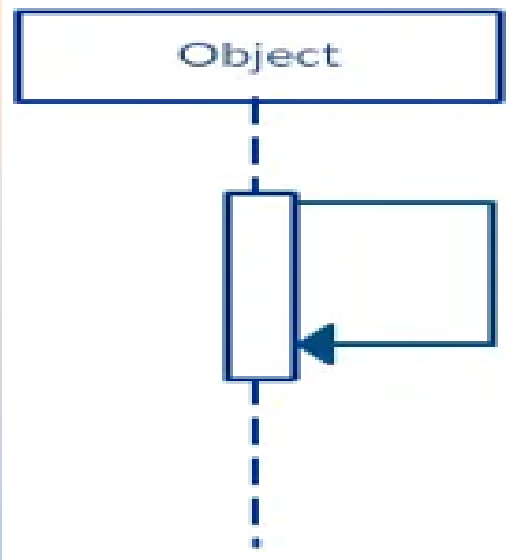
➢ Return Message:

- A return message is used to return the result of an operation to the calling object.
- It is represented by dashed arrow.
- It is only used in response to synchronous messages and it returns of the operation to the calling object.



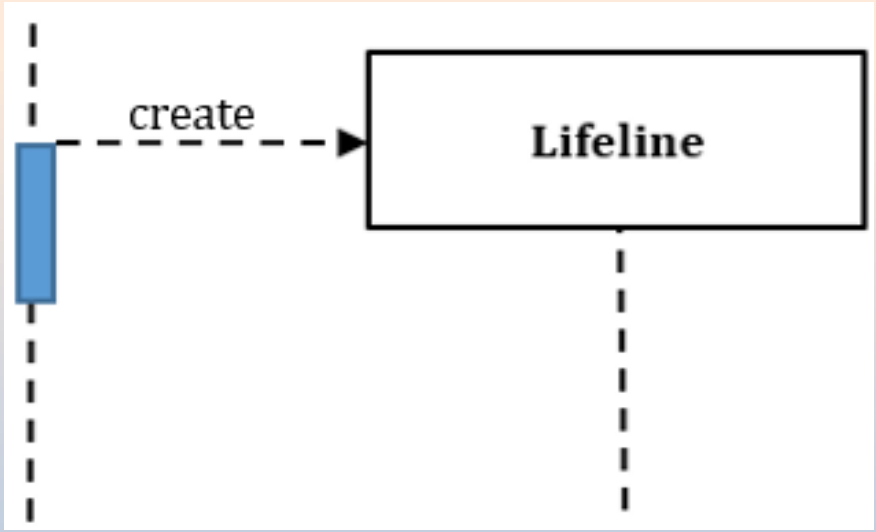
➢ Self Message:

- It represent a message from an object to itself.
- It is represented by a looped arrow.
- Self-messages are often used when an object needs to perform an internal operation or to trigger another operation within itself.



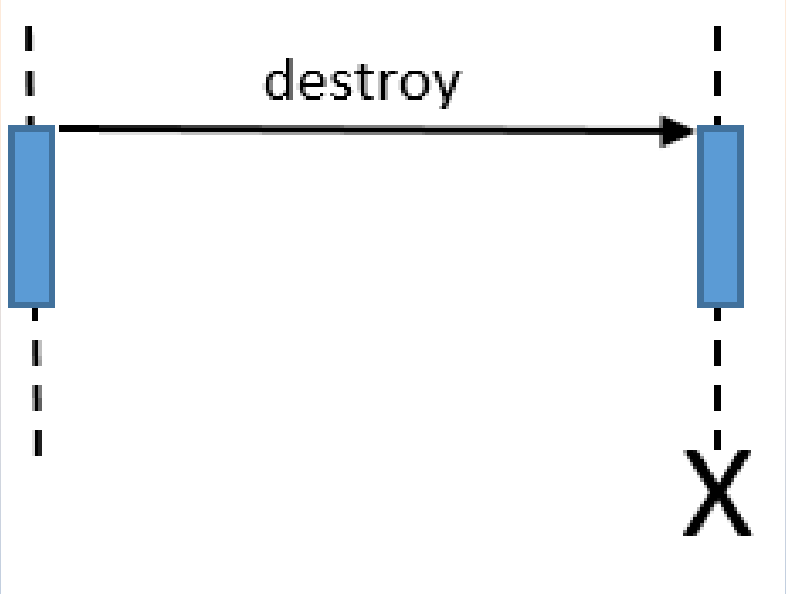
➢ Create Message:

- It is a type of message that represents the creation of a new object or instance of a class in the system.
- It is shown as a dashed line with open arrowhead (looks the same as reply message), and pointing to the created lifeline's head.



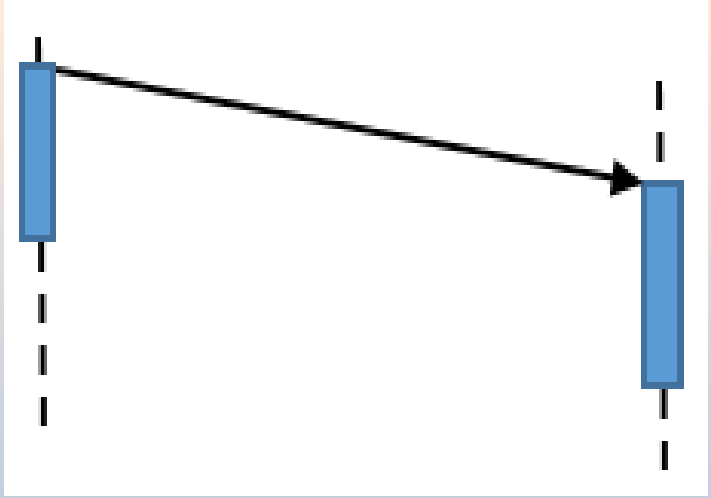
➢ Destroy Message:

- It is a type of a message that represents the deletion or destruction of an object or instance of a class in the system.
- It is represented by an arrow terminating with a X.



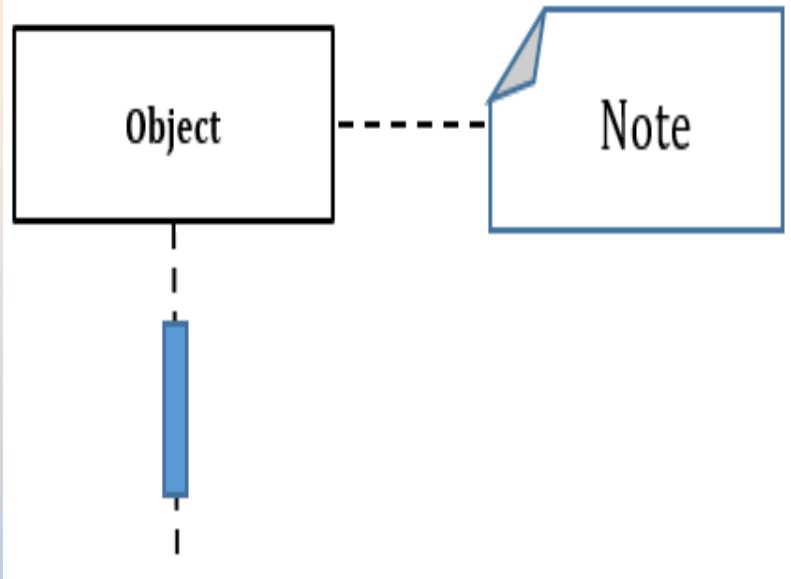
➢ Duration Message:

- It is a type of a message that represents the duration of an activity in the system.
- It is denoted by a horizontal line with an optional label indicating the duration of the activity.

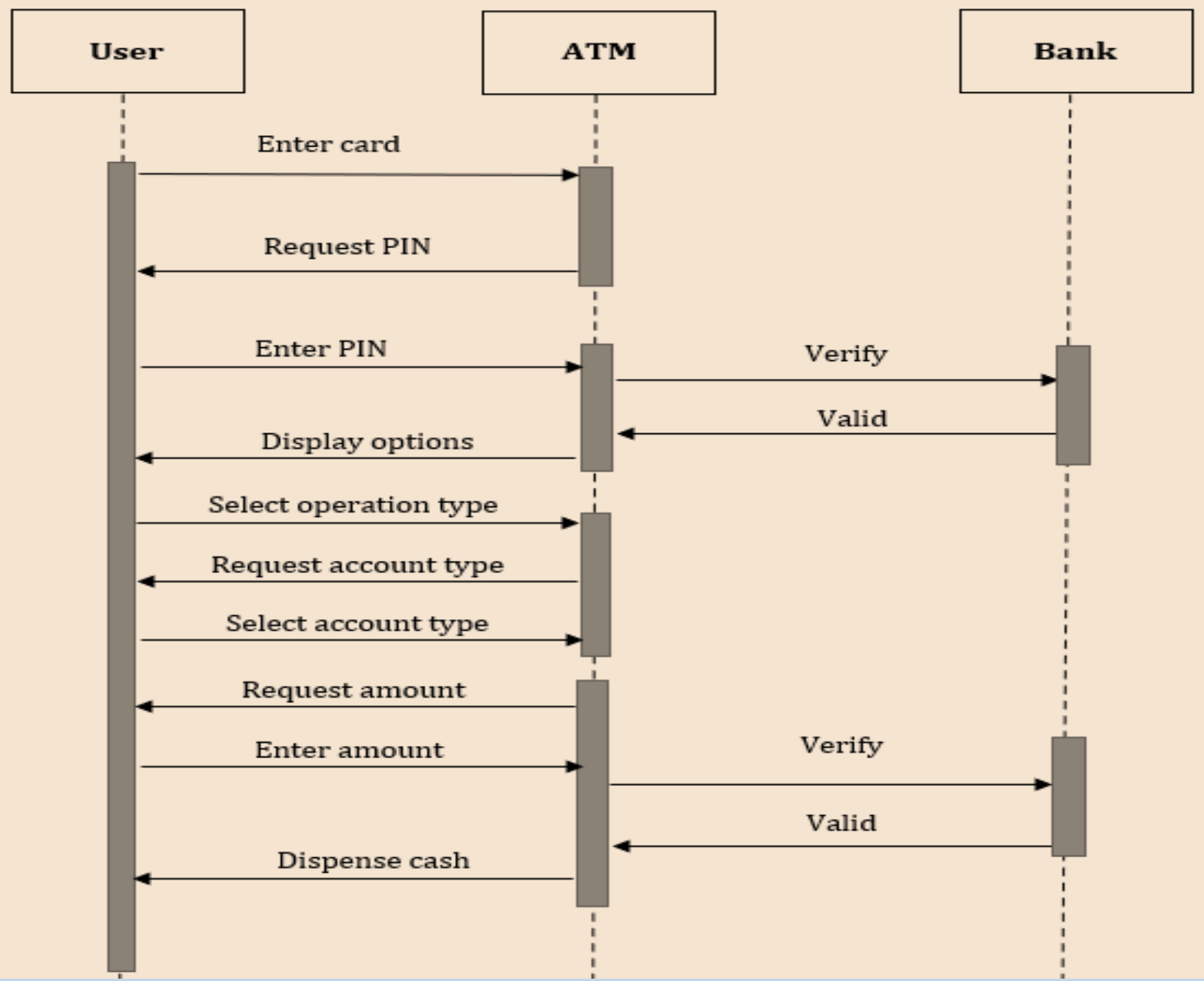


➢ Note or Comment:

- It is a remarks to elements that basically, carries useful information.
- It is represented by a rectangle with a folded corner. And it can be linked with the related object with dashed line.



- Example of Sequence diagram (ATM System : Withdraw Amount):

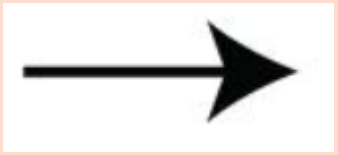
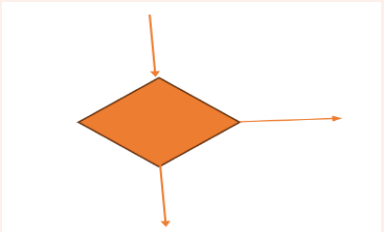


- Advantages of Sequence Diagram:
  - Clear Visualization of Interactions
  - Improved Communication
  - Early Detection of Design Issues
  - Supports Testing and Debugging
- Disadvantages of Sequence Diagram:
  - Complexity for Large Systems
  - Time-Consuming
- Application of Sequence Diagram:
  - Use Case Analysis
  - System Design
  - Documentation
  - Testing

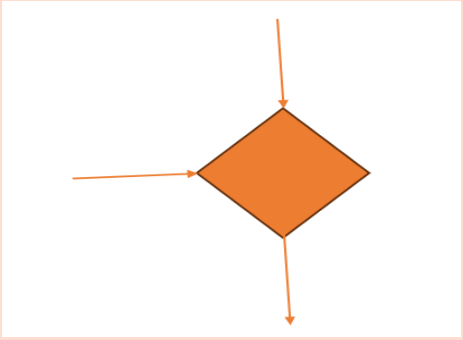
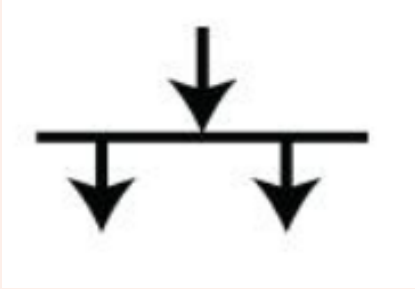
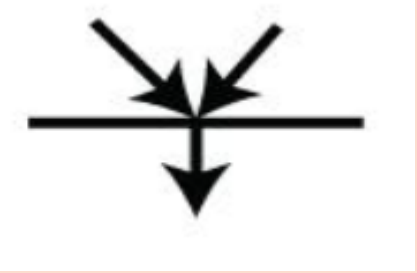
Activity Diagram

- Activity diagram in Software Engineering (SE) is a type of Unified Modeling Language (UML) diagram that visually represents the flow of control or data in a system, It shows the sequence of activities and decisions involved in a process or workflow.
- It consist of activities, states and transitions between activities and states.
- Activity diagrams are similar to procedural flow charts but the difference is that activity diagram support parallel activities.
- An interesting feature of the activity diagrams is the swim lanes, It enable you to group activities based on who performing them, so swim lanes subdivide activities based on responsibilities.
- Activity diagram normally used in business process modeling.

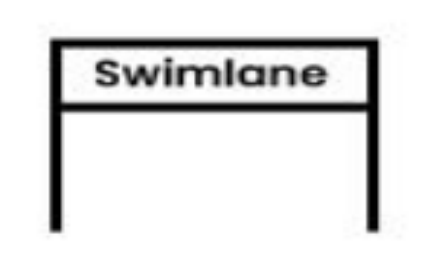
- Component of Activity Diagram:

Component	Description	Symbol
Activity	It represents a particular action taken in the flow of control.	
Start (Initial State) /Stop (Final State)	It is used show start or stop of an activity diagram.	
Flow or Transition	Used to show transfer of control from one activity to another.	
Decision (Branch)	It is a branch where single transition enters and several outgoing transitions.	

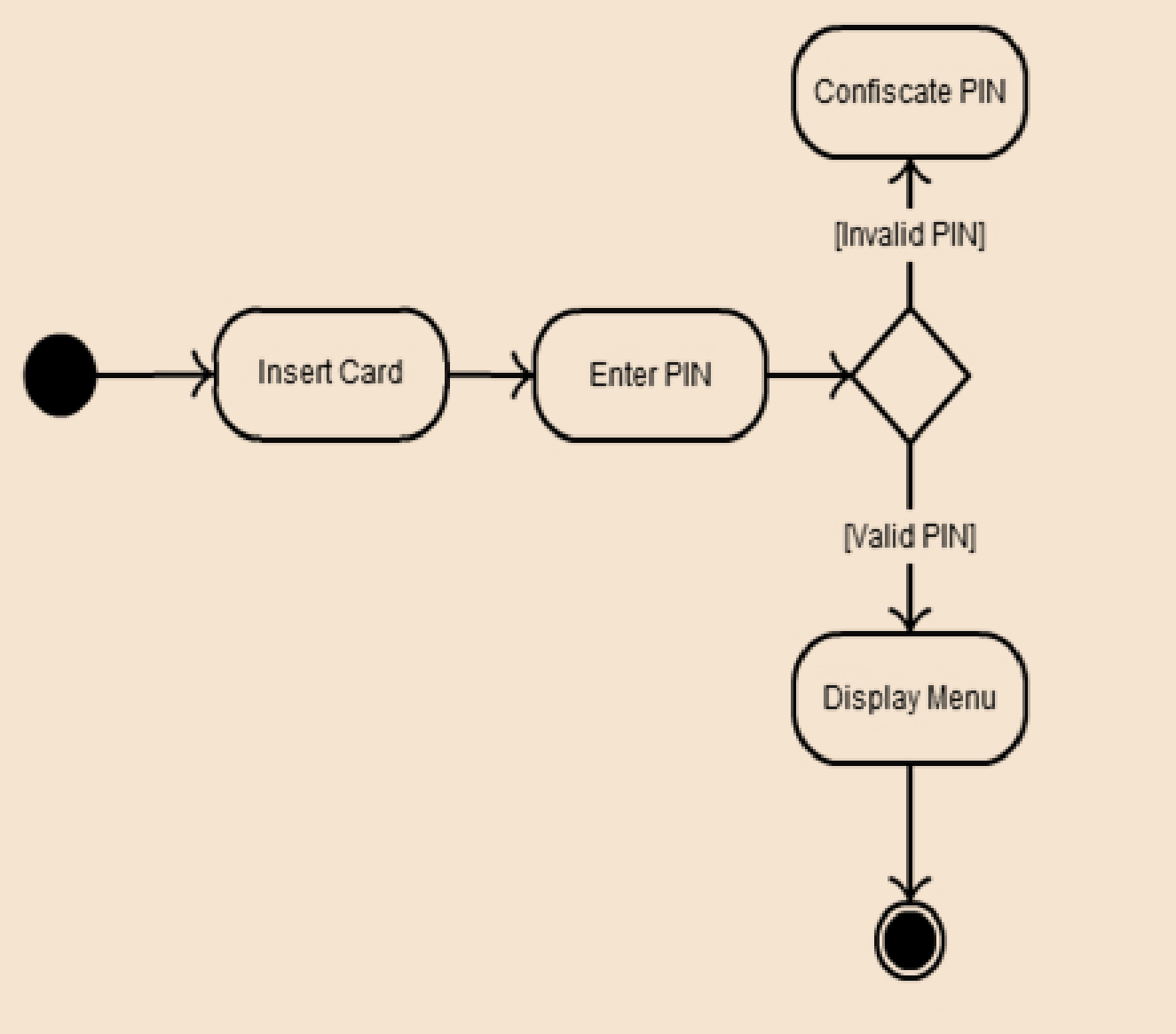
- Component of Activity Diagram:

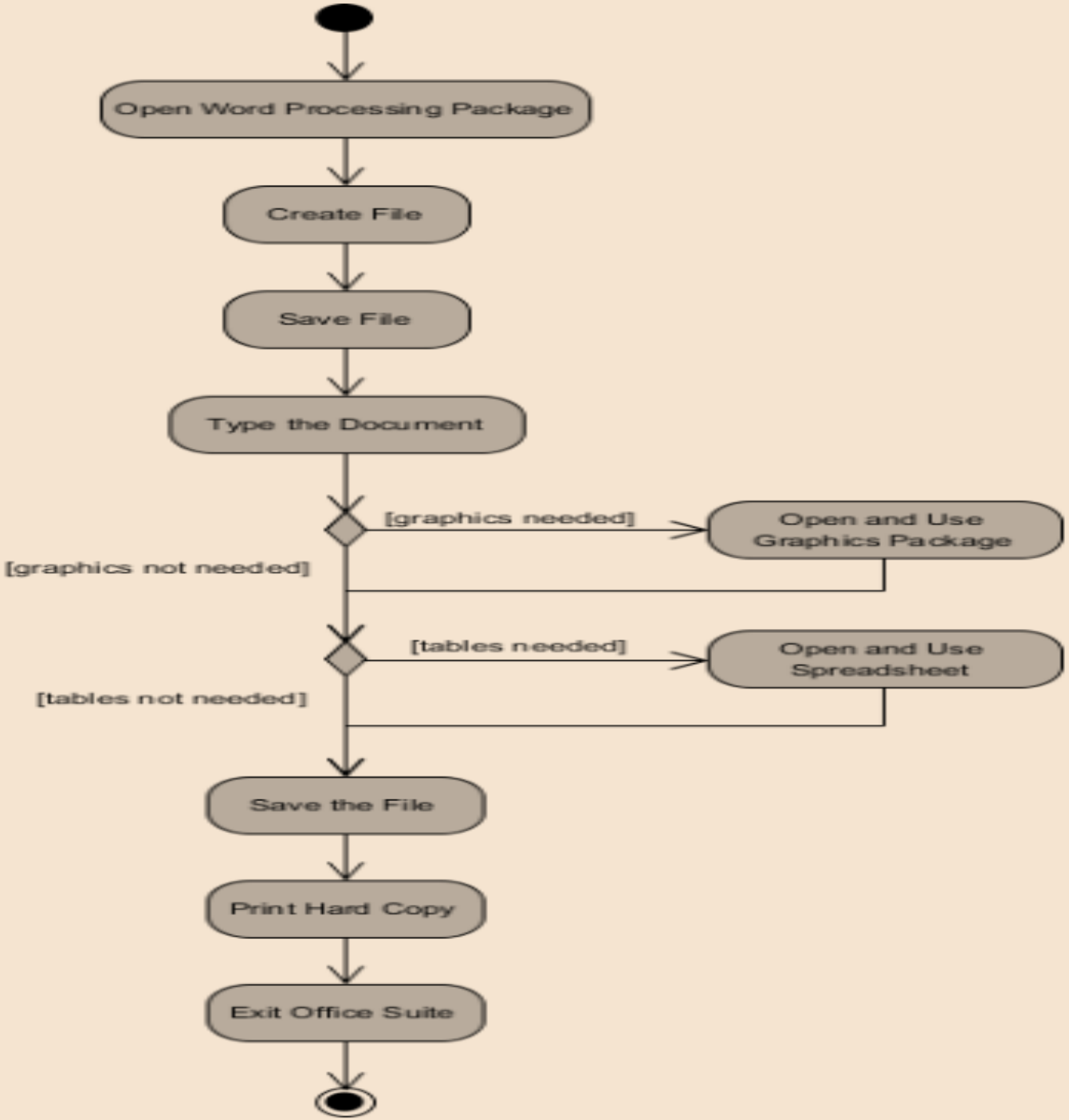
Component	Description	Symbol
Merge	At this point multiple transitions enters and single outgoing transitions occurs.	
Fork	Fork is a point where parallel activities begin. Fork is denoted by black bar with one incoming transition and several outgoing transition.	
Join	Join is denoted by a black bar with multiple incoming transitions and single outgoing transition. It represents the synchronization of all concurrent activities.	

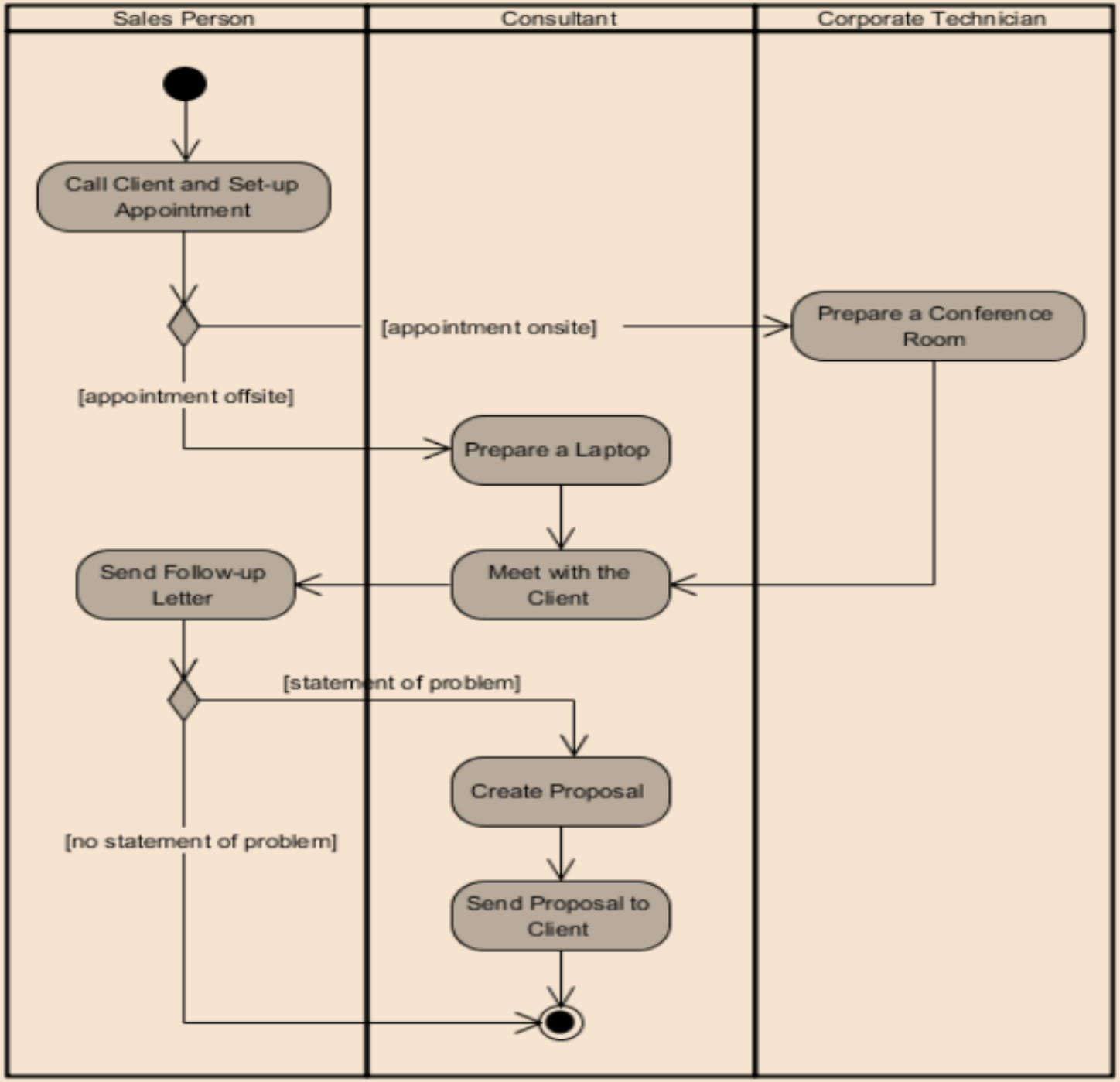
- Component of Activity Diagram:

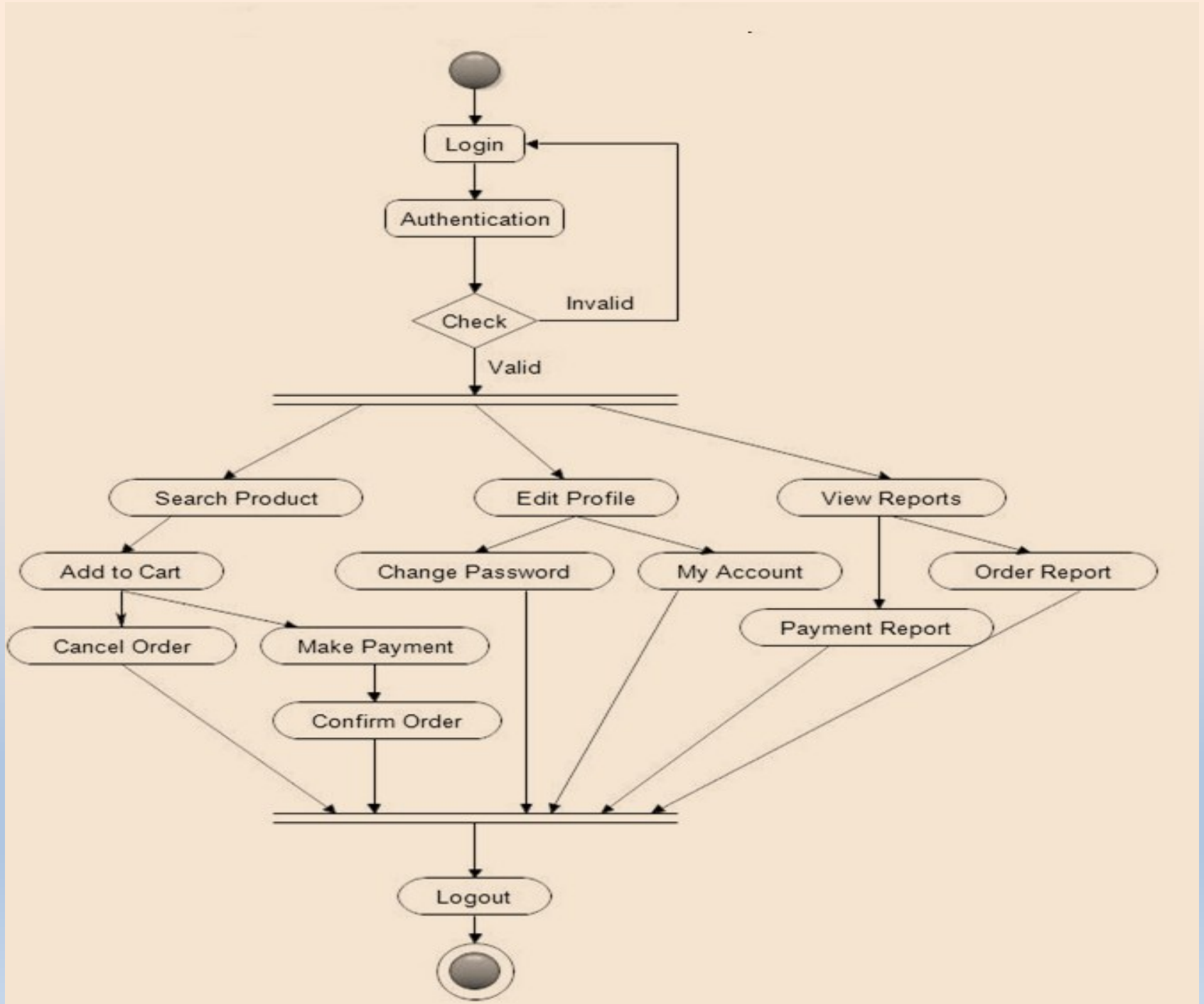
Component	Description	Symbol
Partition / Swim lanes	Different components of an activity diagram can be logically grouped into different areas, called partition or swim lanes. It is denoted by drawing vertical parallel lines.	
Guard Conditions	It controls transition from based on conditions	[ ]

- Activity Diagram of ATM System - Card Validation :




Open Word Processing Package
Create File
Save File
Type the Document
[graphics needed]
Open and Use Graphics Package
[graphics not needed]
[tables needed]
Open and Use Spreadsheet
[tables not needed]
Save the File
Print Hard Copy
Exit Office Suite


Sales Person
Consultant
Corporate Technician
Call Client and Set-up Appointment
[appointment onsite]
Prepare a Conference Room
[appointment offsite]
Prepare a Laptop
Send Follow-up Letter
Meet with the Client
[statement of problem]
Create Proposal
[no statement of problem]
Send Proposal to Client


Login
Authentication
Check
Invalid
Valid
Search Product
Edit Profile
View Reports
Add to Cart
Change Password
My Account
Order Report
Payment Report
Cancel Order
Make Payment
Confirm Order
Logout

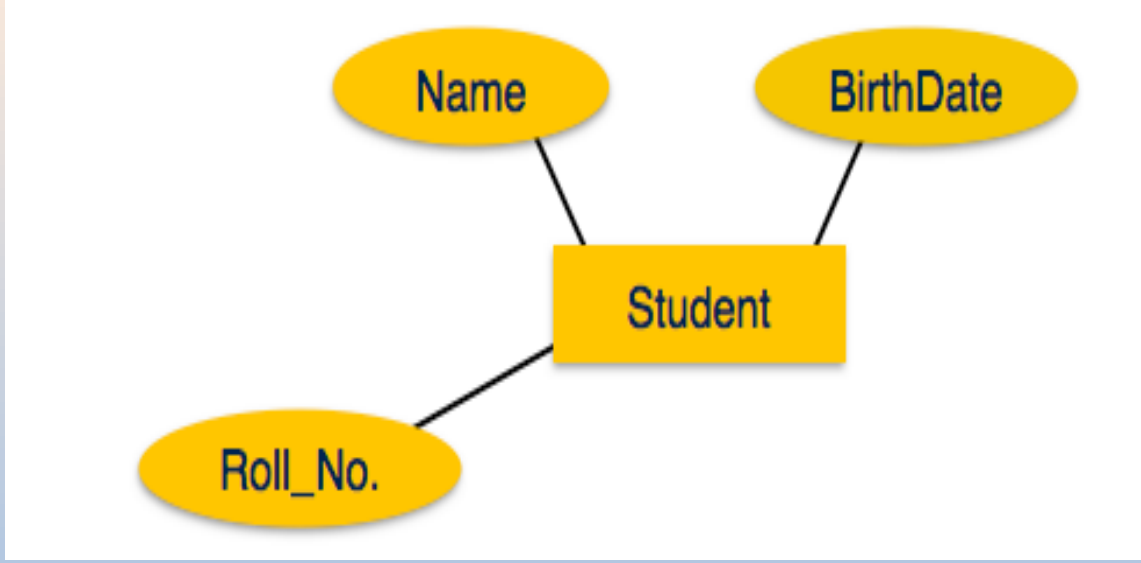
- Advantages of Activity Diagram:
  - Clarity and Understanding for Non-Technical Person also
  - Different activities are grouped together based on actor.
  - Flexibility
- Disadvantages of Activity Diagram:
  - The activity diagram does not provide message part, means do not show any message flow from one activity to another.
  - It can't describe how objects collaborate.
  - Time Consuming
- Applications of Activity Diagram:
  - Business Process Modeling
  - Software Development
  - System Design
  - Project Management
  - Documentation and Training

ER Diagram(Data Modeling Concepts)

- Entity-Relationship (ER) Diagram is a visual representation of the relationships between different entities in a database, It is widely used in database design to model the logical structure of a system.
- ER diagram enables a software engineer to identify data objects and their relationships using graphical notations.
- It has three Main Elements:
  - Data Object (Entity)
  - Attributes
  - Relationships.

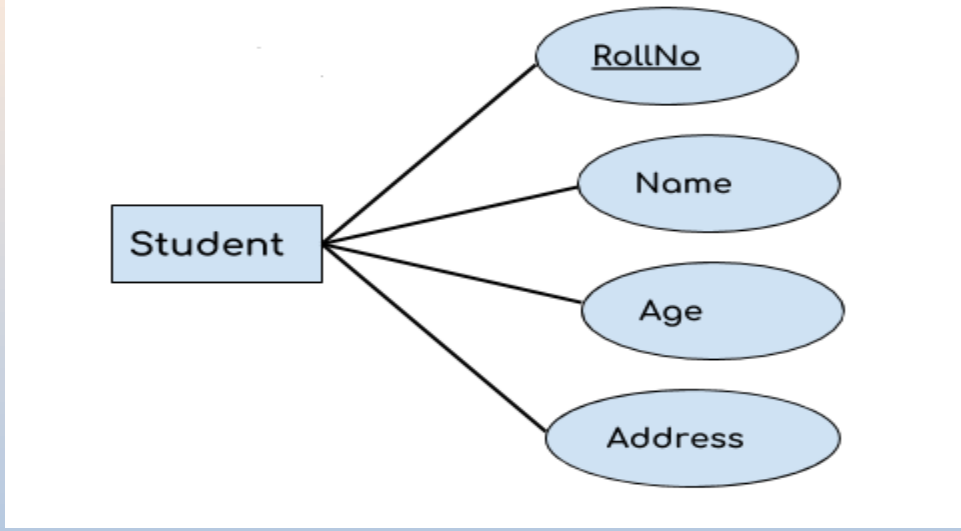
- Data Objects (Entity):
  - A data object is a real-world entity or thing, entity represents a thing that has meaning and about which you want to store or record data.
  - It can be external entity, a thing, an organization, a place or an event.
  - An entity is represented as rectangle in an ER diagram and preferably written in capital letters.
  - Example:

- Attributes
  - An Attribute is a property or characteristic of an entity which provide meaning to the objects.
  - Attributes are represented by means of ellipses. Every ellipse represents one attribute and is directly connected to its entity (rectangle).
  - Example:



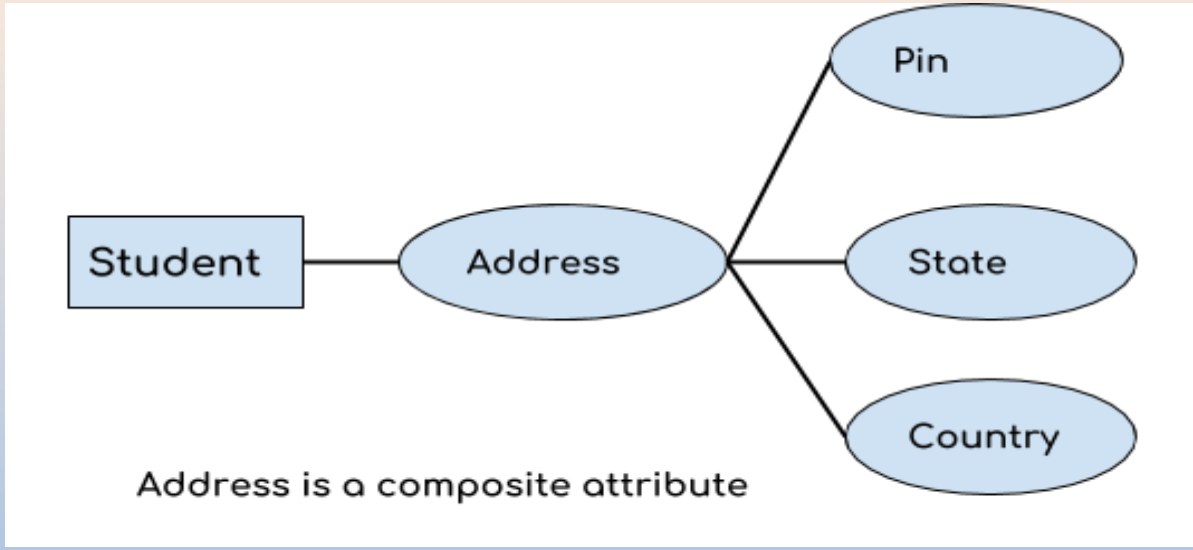
– Types of Attribute: Key Attributes

- A key attribute can uniquely identify an entity from an entity set.
- For example, student roll number can uniquely identify a student from a set of students.
- Key attribute is represented by oval same as other attributes however the text of key attribute is underlined.



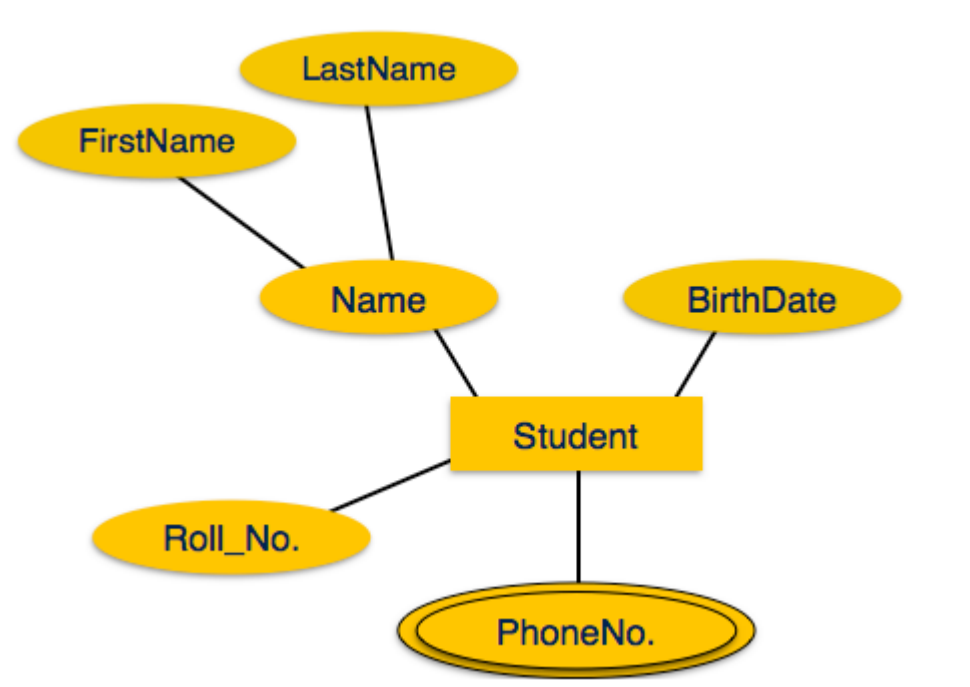
– Types of Attribute: Composite Attributes

- An attribute that is a combination of other attributes is known as composite attribute.
- For example, In student entity, the student address is a composite attribute as an address is composed of other attributes such as pin code, state, country.



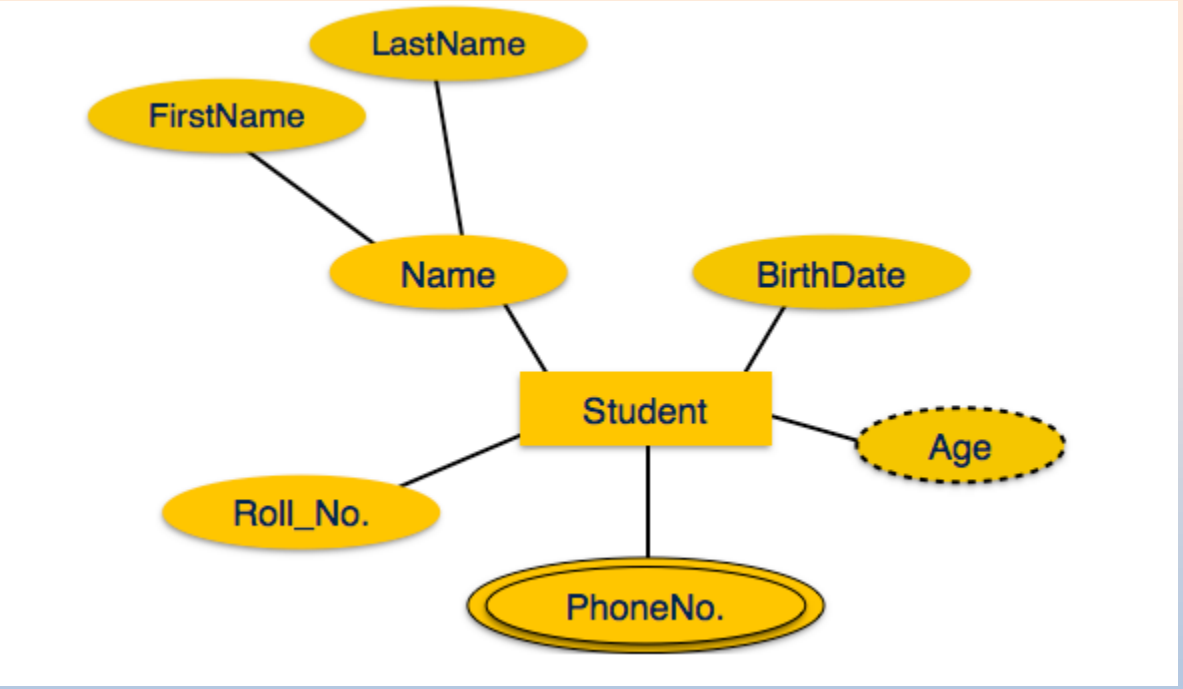
– Types of Attribute: Multivalued Attributes

- An attribute that can hold multiple values is known as Multivalued attribute.
- It is represented with double ovals in an ER Diagram.

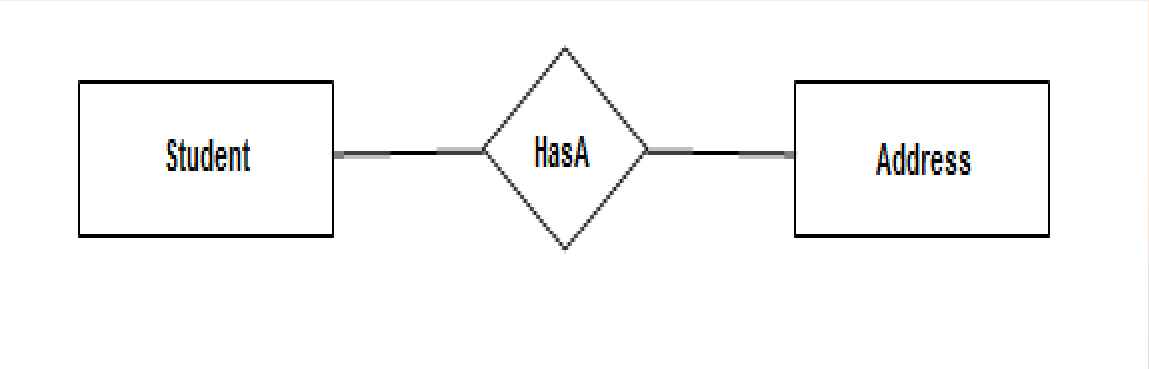


– Types of Attribute: Derived Attributes

- A derived attribute is one whose value is dynamic and derived from another attribute.
- It is represented by dashed oval in an ER Diagram.

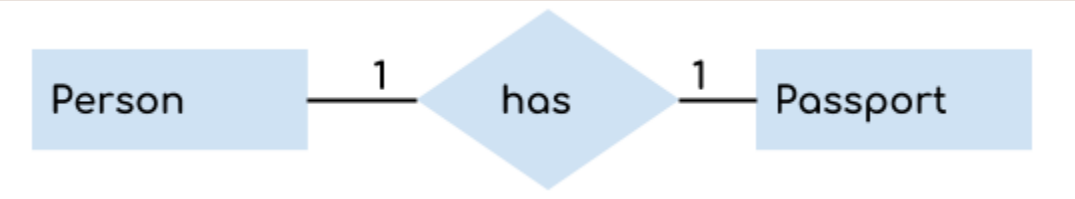


- Relationship
  - It shows the relationship among entities.
  - Relationship is represented using diamond shape symbol with joined relationship name.



- Cardinality
  - Cardinality is the number of instance of an entity from a relation that can be associated with the relation.
  - Different Cardinalities are explained below:

1. One to One (1 : 1)
   - A single instance of an entity is associated with a single instance of another entity

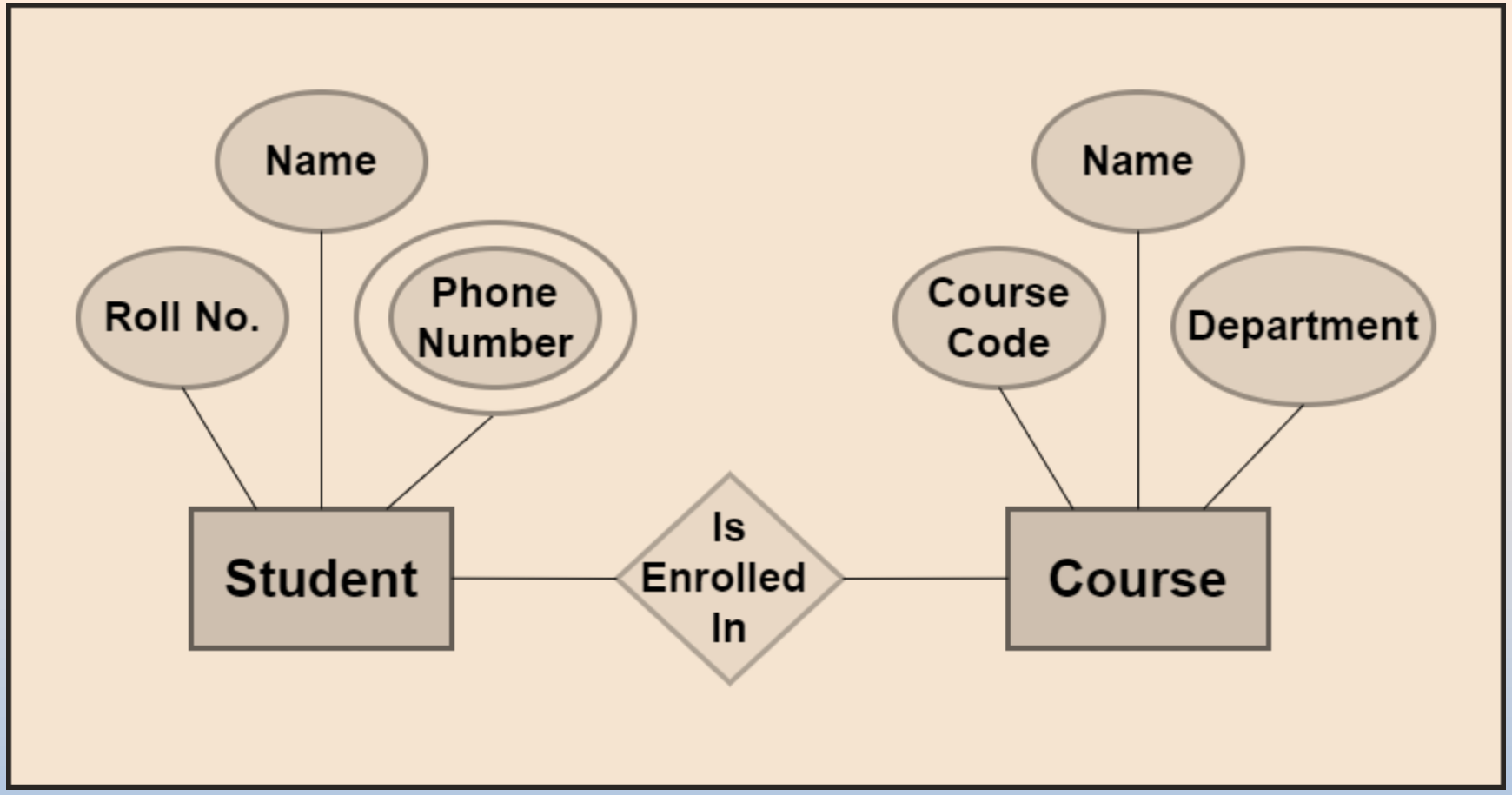


2. One to Many (1 : M)
   - A single instance of an entity is associated with more than one instances of another entity

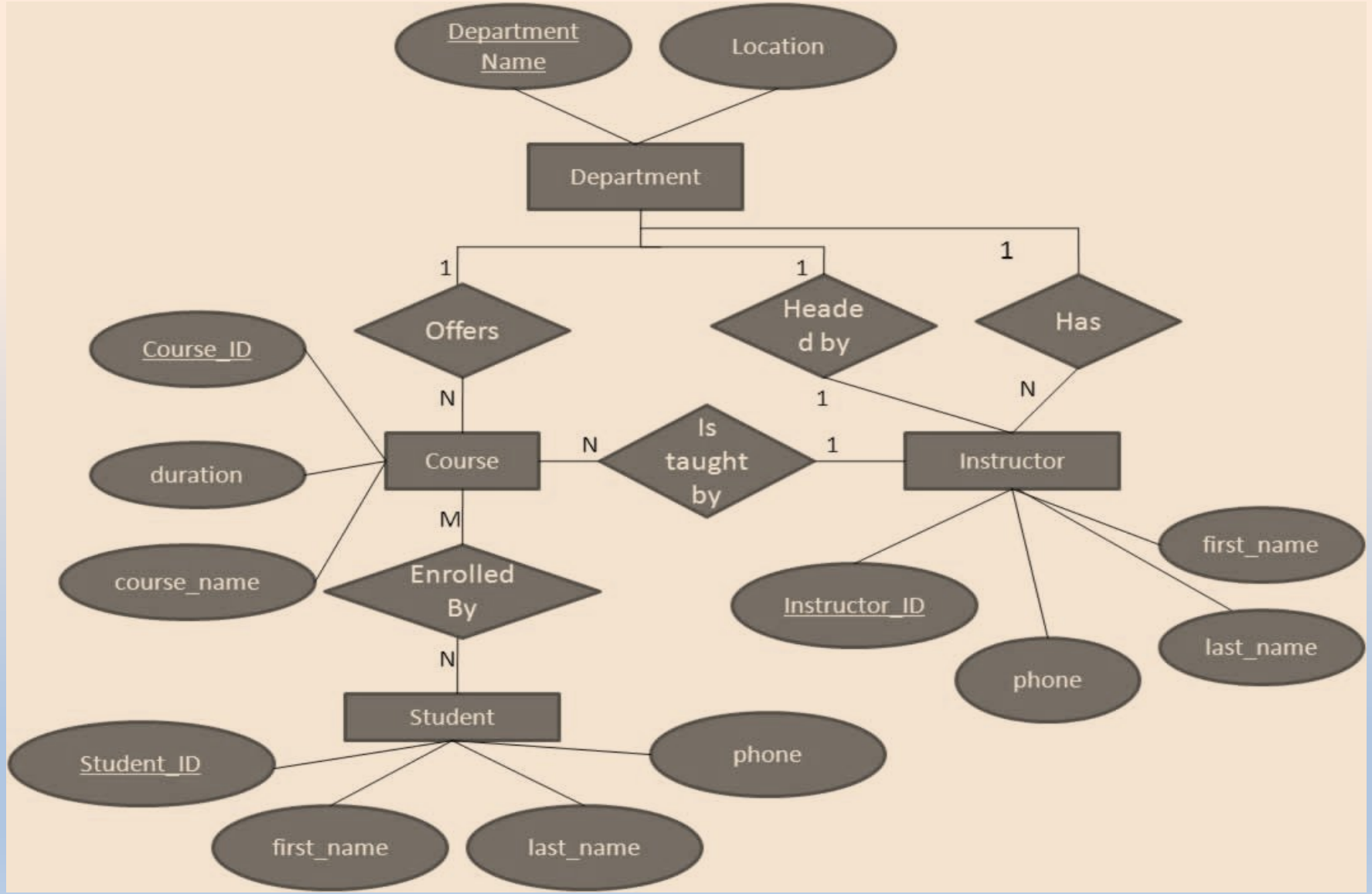
3. Many to One (M : 1)
   - More than one instances of an entity is associated with a single instance of another entity.

4. Many to Many (M:M)
   - More than one instances of an entity is associated with more than one instances of another entity.

- Example:


Name
Roll No.
Phone Number
Student
Is Enrolled In
Course
Name
Course Code
Department

- Example:


Department Name
Location
Department
1
1
1
Offers
Heade d by
Has
Course_ID
N
1
N
duration
Course
N
Is taught by
1
Instructor
M
course_name
Enrolled By
first_name
Instructor_ID
last_name
phone
N
Student
Student_ID
phone
first_name
last_name

GTU Questions

Unit-2 (CO-2)	
Sr No.	**Question**
1	State the requirement-gathering phase of software development OR List requirement gathering techniques, explain any two.
2	Explain Software Requirement Specification (SRS) with its components. OR List any eight characteristics of Software Requirement specification OR List different characteristics of a good SRS.
3	Write a difference between functional and Non-Functional requirements. OR Prepare any three functional requirements for the Bus ticket reservation system OR Prepare customer requirements for a college management system. OR Prepare system functional requirement for online shopping system. OR List Various functional requirements of Hospital Management System. OR Prepare system functional requirement for Library management system
4	Differentiate analysis and design.
5	Explain Cohesion with their classification. OR Define cohesion. Explain classification of cohesion. OR Explain what is cohesion? Enlist it's all types and explain any two type of cohesion with example
6	Explain what is coupling? Enlist it's all types and explain any two type with example
7	Explain the Data Flow Diagram with its primitive symbols and example. OR Draw context diagram and level 1-DFD for Library management system. OR List the advantages of using the DFD model. OR Draw and explain context diagram and level-1 DFD for Online shopping system. OR Explain primitive symbols used in DFD.
8	Explain the component of the use case diagram. OR Define use case diagram. Explain it with example. OR Explain Use Case diagram with example. OR Explain Components of Use Case Diagram
9	Explain different relationship in class diagram. OR Draw the class diagram for Attendance Management System.
10	Write a short note on components of sequence diagram.
11	Explain Activity diagram with example. OR Draw the Activity diagram for ATM (Automated teller machine) System. OR Draw activity diagram for online shopping system.